Escuela Politécnica Superior

Aplicación de redes neuronales para la predicción de un reactor continuo de tanque agitado con controlador PI

Grado en Ingeniería Química

Trabajo Fin de Grado

Autor:

Rodrigo Carvajal Pérez

Tutor/es:

José Rubén Ruíz Femenia

Juan Javaloyes Antón

29 de mayo de 2026

Universitat d'Alacant
Universidad de Alicante

AGRADECIMIENTOS

Quiero empezar dando las gracias a todo el profesorado que me ha impartido clase y aportado conocimientos durante mi estancia en la Universidad de Alicante.

En especial a mi tutor, Rubén Ruíz, por el apoyo constante durante la realización de este proyecto, su amabilidad, y por fomentar un cambio dirigido hacia el aprendizaje de técnicas de vital importancia y de plena actualidad, como con la propuesta de este proyecto. Mencionar también a Juan Javaloyes, cotutor de este proyecto, por la ayuda ofrecida.

Doy gracias también a aquéllos que me han acompañado durante el grado y me han aportado ayuda y momentos muy felices. Amigos de Elche, Alicante y Glasgow, a todos, gracias.

Y por último, quiero agradecer enormemente a mi familia, sin ellos, nada sería posible. A mi Madre, por su sacrificio, cariño, y fortaleza en los momentos de mayor debilidad.

Índice

1. INTRODUCCIÓN **11**

2. DESCRIPCIÓN DEL CASO DE ESTUDIO **12**

2.1. Descripción del Reactor 12

2.1.1. Parámetros constantes del reactor 13

2.1.2. Ecuaciones del RCTA 14

2.1.3. Ecuaciones del controlador PI 15

2.2. Cálculo y elección del estado estacionario 15

2.3. Comportamiento del reactor 18

2.3.1. Evolución de las concentraciones 19

2.3.2. Efecto de la temperatura 20

2.4. Comportamiento del reactor con controlador PI 24

3. OBTENCIÓN DE DATOS PARA EL ENTRENAMIENTO DE UNA RED NEURONAL. **28**

3.1. Rango de los parámetros de los controladores PI. 28

3.2. Muestreo de datos 29

3.3. Filtrado de simulaciones dentro del Dataset 30

3.4. Estructura del Dataset. 31

3.5. Análisis de las muestras filtradas. 32

3.5.1. Espacio ocupado de las ganancias de los PIs. 33

3.5.2. Gráficos de variabilidad. 36

4. CONSTRUCCIÓN DE LA RED NEURONAL **40**

4.1. Datos de entrenamiento y datos de prueba. 41

4.2. Normalización 41

4.3. Definición del modelo. 42

4.4. Configuración del modelo. 45

4.5. Entrenamiento del modelo. 47

4.6. Evaluación con los datos de prueba. 50

5. RESULTADOS. 51

6. LIMITACIONES Y FUTURAS MEJORAS. 60

7. CONCLUSIONES. 64

APÉNDICE

A. CÁLCULO ESTADOS ESTACIONARIOS 67

B. ESTUDIO DEL COMPORTAMIENTO DEL REACTOR. 71

B.1. Estudio general. 71

B.2. Estudio de la Temperatura. 72

C. COMPORTAMIENTO DEL REACTOR CON CONTROLADOR PI 74

D. OBTENCIÓN DE DATOS PARA EL ENTRENAMIENTO DE UNA RED NEURONAL. 79

D.1. Muestreo cambiando los parámetros del controlador 79

D.2. Espacio ocupado de los parámetros del controlador. 84

D.3. Filtrado y descarte 85

D.4. Cambio de la estructura del Dataset. 88

D.5. Espacio ocupado de los parámetros del controlador. 89

D.6. Gráficos de variabilidad 91

E. RED NEURONAL 93

E.1. Importación de librerías y cargado del Dataset 93

E.2. Separación del Dataset en datos de entrenamiento y de prueba 94

E.3. Normalización 94

E.4. Definición del modelo. 96

E.5. Configuración del modelo. 97

E.6. Entrenamiento del modelo. 97

E.7. Evaluación con los datos de prueba. 98

E.8. Predicciones. 98

Índice de figuras

1. Esquema del reactor continuo de tanque agitado. 13
2. Sensibilidad de la concentración de C_B respecto a la temperatura de la camisa $T_{c,s}$. 17
3. $C_{B,s}$ frente a $T_{c,s}$. 18
4. Evolución de las concentraciones. 20
5. Evolución de la temperatura. 21
6. Contribución de cada término al balance de energía. 22
7. Evolución de T sin intercambio de energía. 23
8. Evolución de los términos del balance de energía sin intercambio de calor. 24
9. Control de las concentraciones. 26
10. Control del volumen. 26
11. Control de la temperatura. 27
12. Control de las variable manipuladas. 27
13. Control de las variable manipuladas. 28
14. Distribución de los parámetros de los controladores generados mediante muestreo Sobol. A la derecha para controlador 1 y a la izquierda para el 2. 33
15. Distribución de los parámetros de los controladores generados mediante muestreo Sobol. A la derecha las ganancias y a la izquierda los tiempos integrales. 34
16. Comparativa de simulaciones guardadas y descartadas por medio de los parámetros del controlador 1. 34
17. Comparativa de simulaciones guardadas y descartadas por medio de los parámetros del controlador 2. 35
18. Comparativa de simulaciones guardadas y descartadas por medio de los parámetros cruzados. 36
19. Rangos de simulaciones y std para C_B. 37
20. 5 simulaciones de C_B. 38
21. Rangos de simulaciones y std para T. 38
22. 5 simulaciones de T. 39
23. Rangos de simulaciones y std para V. 39

24. 5 simulaciones de V. 40
25. Esquema de la red neuronal. 43
26. Esquema de cada neurona. 44
27. Esquema del descenso de gradiente. 46
28. Esquema de la separación de datos realizada. 48
29. Progersión del error por época entre datos de entrenamiento y validación. 48
30. Últimas 3 épocas. 49
31. Dinámicas de C_B y T. Simulación 109. 52
32. Dinámica de V. Simulación 109. 52
33. Dinámicas de C_B y T. Simulación 75. 53
34. Dinámica de V. Simulación 75. 53
35. Dinámicas de C_B y T. Simulación 57. 54
36. Dinámica de V. Simulación 57. 54
37. Dinámicas de C_B y T. Simulación 125. 55
38. Dinámica de V. Simulación 125. 55
39. Dinámicas de C_B y T. Simulación 218. 56
40. Dinámica de V. Simulación 218. 56
41. Dinámicas de C_B y T. Simulación 243. 57
42. Dinámica de V. Simulación 243. 57
43. Dinámicas de C_B y T. Simulación 208. 58
44. Dinámica de V. Simulación 208. 58
45. Dinámicas de C_B y T. Simulación 208. 59
46. Dinámica de V. Simulación 251. 59

Índice de tablas

1. Parámetros del reactor 14
2. Estado estacionario escogido 18
3. Parámetros del controlador 24
4. Rangos de variación de los parámetros de los controladores PI 29
5. Estructura esquemática del conjunto de datos 31
6. Estructura esquemática del conjunto de datos en formato ancho 32
7. Estadísticas descriptivas de las variables de entrada normalizadas. . . . 42
8. Resumen de la arquitectura de la red neuronal 45
9. Hiperparámetros utilizados en la red neuronal y posibles alternativas viables. 61
10. Otros hiperparámetros potencialmente útiles para la red neuronal. . . . 63

RESUMEN

Para la creación de una red neuronal que prediga el comportamiento de un RCTA con un PI implementado, primero se ha llevado a cabo un estudio del comportamiento de dicho reactor, con la finalidad de conocer cómo varían las variables con el transcurso del tiempo. Luego, se ha llevado a cabo un muestreo, llevando a cabo simulaciones con combinaciones distintas de los parámetros del controlador. Seguidamente se explica cómo se recogen y filtran los datos de las simulaciones en un documento de datos, discutiendo además, cómo interfiere la estructura de este documento en el entrenamiento y predicción del modelo creado. Por último, se construye la red neuronal y se predicen las dinámicas de la concentración del producto deseado, la temperatura interna del reactor, y el volumen del mismo, objetivo principal de este proyecto.

1. INTRODUCCIÓN

El avance en el desarrollo de la inteligencia artificial es fréntico, y la etapa en la que empieza a ganar popularidad por su gran potencial ya ha pasado, dando lugar en el mundo tecnológico a otra en la que se está consolidando firmemente en casi cualquier ámbito, siendo su conocimiento un requisito importante para aquellos ingenieros que deseen tener un perfil adaptado a la actualidad y mantenerse competitivos. Sin duda, la inteligencia artificial marca un cambio de época, una revolución a la altura de la creación de la imprenta moderna en 1440, o incluso del dominio del fuego.

Existen dos tipos de inteligencia artificial, la débil, y la fuerte. La débil, una de sus definiciones más aceptadas, es ”el esfuerzo para automatizar tareas intelectuales normalmente realizadas por humanos”. Y ésta es en la que verdaderamente ha habido una revolución. La inteligencia artificial fuerte, es aquella que considera que las máquinas pueden ser superinteligentes y sustuir al hombre por completo. Esta útlima es más bien un acto de fe que una potencial posibilidad, ya que, aunque algunos investigadores estén trabajando en ella, los avances son mínimos, y podrían pasar muchas décadas hasta que verdaderamente se haya creado una inteligencia igual o superior a la del ser humano. Aún así, puede que el devenir de los acontecimientos sea sorprendente, y descartar la posibilidad que pueda finalmente desarrollarse, sería ingenuo.

Existe un subcampo de la inteligencia artifical llamado Machine Learning, que proporciona a los ordenadores la capacidad de aprender sin ser explícitamente programados. Generalizando, se podría decir que consiste en desarrollar un algoritmo de predicción para cada problema. Estos algoritmos, aprenden de los datos con el fin de encontrar patrones para comprender qué quieren decir esos datos, y así, construir un modelo para predecir o clasificar elementos.

Machine Learning puede ofrecer distintos tipos de algoritmos. La categoría que se usará en el presente proyecto es el conocido como aprendizaje supervisado (supervised learning), en el que los datos que se usan para el entrenamiento, incluyen la solución deseada, llamada etiqueta (label). Si el entrenamiento es óptimo, luego del aprendizaje supervisado, se obtiene un modelo capaz de generalizar datos correctamente y hacer predicciones a partir de datos de entrada que aún no ha visto.

Un caso especial de algoritmos de Machine Learning, son las redes neuronales artificiales, que, en cierto modo, tratan de imitar las actividad de las capas de neuronas en la neocorteza del cerebro, parte de él donde ocurre el pensamiento humano. A esto es lo que se conoce como Deep Leanring, y se basará también en el aprendizaje supervisado. Una de sus mayores ventajas, es la capacidad de aprender automáticamente la representación de características en múltiples niveles de abstracción, permitiendo que, por ejemplo,

pueda sustituir a funciones complejas tradicionales que describan la realidad, en el caso de este proyecto, de un reactor.

Con esto, se da por introducido este Trabajo de Fin de Grado, y se puede dar paso a la descripción del problema.

2. DESCRIPCIÓN DEL CASO DE ESTUDIO

El modelo de redes neuronales, el cual se presentará más adelante en este texto, tiene como objetivo aprender de un reactor continuo de tanque agitado (RCTA) con un controlador PI implementado y predecir sus valores.

Cabe ir desglosando pormenorizadamente el procedimiento empleado hasta llegar a la construcción de la redes neuronal. En este apartado, se pretende introducir el caso de estudio en cuestión y los pasos seguidos para dicho propósito.

Primero, se calcularán los valores del estado estacionario del RCTA, por medio de la resolución de las ecuaciones ordinarias diferenciales (EDOs) que describen al reactor, considerando que no hay acumulación. Luego, se simulará el RCTA, aún sin controlador, con tal de conocer cuál es su comportamiento en el transcurso del tiempo. Por último, se añadirá el controlador PI a las simulaciones, de tal forma que las variables alcancen el estado estacionario calculado.

Todo esto, aparte de estudiar cómo actúa el RCTA con, y sin PI, tiene como finalidad la creación de un código con el que posteriormente se haga un muestreo variando distintos parámetros, y así, crear un documento de datos (Dataset) amplio y variado para propiciar un buen entrenamiento de la red neuronal, dentro de los rangos en los que tienen sentido físico. Se verá en el apartado 3.

2.1. Descripción del Reactor

El reactor, como se ha mencionado previamente, es un RCTA (Bloor et al., 2025, p. 6) en el que se lleva a cabo la siguiente reacción:

$$A \rightarrow B \rightarrow C$$

La reacción de A para dar B será la reacción 'a' y la reacción de B a C se denominará 'b'. Las velocidades de reacción serán: r_a y r_b (mol/m^3/min) respectivamente, siendo

sus expresiones las mostradas a continuación:

$$r_a = k_a e^{-E_a/(RT)} C_A \tag{1}$$

$$r_b = k_b e^{-E_b/(RT)} C_B \tag{2}$$

Tiene incorporado un PI para controlar el volumen del reactor: V (m^3), la temperatura interna: T (K) y la concentración de B: C_B (mol/m^3), siendo éste el compuesto deseado.

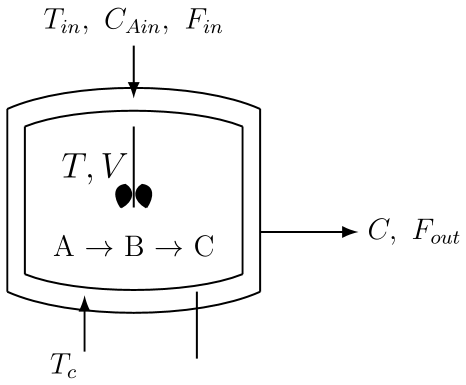


Figura 1: Esquema del reactor continuo de tanque agitado.

La Figura 1 muestra un esquema del RCTA, en donde se aprecia la corriente de entrada a la temperatura T_{in} (K), la concentración de entrada C_{Ain} (mol/m^3) y el caudal volumétrico de entrada F_{in} (m^3/s). Además, el reactor está envuelto por un fluido de intercambio de energía que fluye a través de una camisa con caudal constante a la temperatura T_c (más adelante se comprobará si este fluido es calefactor o refrigerante). Por último, la Figura 1 muestra la corriente de salida F_{out} (m^3/s) y C, que representa las concentraciones de los 3 compuestos: C_A, C_B, y C_c.

2.1.1. Parámetros constantes del reactor

A continuación, se muestra una tabla en la que se recogen aquellos valores que se consideran constantes en las simulaciones para la descripción del reactor. En el caso en el que alguno de estos valores cambie, se especificará y se justificará debidamente dicho cambio en el presente texto.

Tabla 1: Parámetros del reactor

Parámetro	**Valor**	**Unidad**
T_{in}	350	K
$C_{A,in}$	1	$\mathrm{mol\,m^{-3}}$
F_{out}	100	$\mathrm{m^3\,s^{-1}}$
ρ	1000	$\mathrm{kg\,m^{-3}}$
C_p	0.239	$\mathrm{J\,kg^{-1}\,K^{-1}}$
UA	5×10^4	$\mathrm{W\,K^{-1}}$
ΔH_a	5×10^3	$\mathrm{J\,mol^{-1}}$
E_a/R	8750	K
k_a	$7{,}2 \times 10^{10}$	$\mathrm{s^{-1}}$
ΔH_b	4×10^3	$\mathrm{J\,mol^{-1}}$
E_b/R	10750	K
k_b	$8{,}2 \times 10^{10}$	$\mathrm{s^{-1}}$

Siendo ρ la densidad del fluido que fluye dentro del reactor, C_p la capacidad calorífica de la mezcla, U el coeficiente global de transferencia de materia, A el área de transferencia de materia, ΔH_a y ΔH_b las entalpías de las reacciones, E_a y E_b las energías de activación, k_a y k_b los factores preexponenciales de la ley de Arrhenius para las reacciones a y b respectivamente. Por último, R es la constante universal de los gases ($8{,}31\ \mathrm{J\,mol^{-1}\,K^{-1}}$).

2.1.2. Ecuaciones del RCTA

Ahora, se presentan las ecuaciones que describen al reactor, procedentes de balances de materia y energía. Con ellas, se obtendrán las soluciones de las EDOs, lo que permite conocer cuál es el comportamiento del RCTA.

$$\frac{dC_A}{dt} = \frac{F_{in}(C_{A,in} - C_A)}{V} - r_a \tag{3}$$

$$\frac{dC_B}{dt} = r_a - r_b - \frac{F_{in}\,C_B}{V} \tag{4}$$

$$\frac{dC_C}{dt} = r_b - \frac{F_{in}C_C}{V} \tag{5}$$

$$\frac{dT}{dt} = \frac{F_{in}(T_{in} - T)}{V} - \frac{\Delta H_a}{\rho C_p} r_a - \frac{\Delta H_b}{\rho C_p} r_b + \frac{UA(T_c - T)}{V C_p\,\rho} \tag{6}$$

$$\frac{dV}{dt} = F_{in} - F_{out} \tag{7}$$

2.1.3. Ecuaciones del controlador PI

Para que las variables alcancen los estados estacionarios deseados, es preciso el uso de un controlador, en este caso con una parte proporcional y otra integral (PI). La expresión general de los controladores PI es la siguiente:

$$c(t) = K_c\,\varepsilon(t) + \frac{K_c}{\tau_I}\int_0^t \varepsilon(t)\,dt + c_s \tag{8}$$

En donde $c(t)$ es la variable manipulada, K_c es la ganancia del controlador de la parte proporcional, τ_I la ganancia de la parte integral, c_s es el *bias*, que tendrá el valor del estado estacionario de la variable manipulada. A su vez, $\varepsilon(t)$ es el error, estando definido de la siguiente manera:

$$\varepsilon(t) = c_s - c(t) \tag{9}$$

Por tanto, es conveniente mencionar cuáles van a ser las variables controladas y manipuladas y cuáles van a ser las parejas de control, basadas en un estudio de RGA (Bloor et al., 2025, p. 11) en el que se demuestra qué par de variables dará mejores resultados, entendiéndose como buenos resultados aquellos que satisfactoriamente alcancen el estado estacionario deseado.

Las variables controladas son: C_B, T y V. A su vez, las variables manipuladas son: T_c y F_{in}. T_c controlará a C_B, y F_{in} controlará a V. T se controlará por medio de la regulación de las dos variables manipuladas, no siendo necesaria una ecuación de controlador para la temperatura interna del reactor. Por tanto, habrá dos ecuaciones que representen dos controladores PI, uno para cada lazo de control, siendo sus ecuaciones las siguientes:

$$T_c(t) = K_{c,1}\,(C_{B,s} - C_B(t)) + \frac{K_{c,1}}{\tau_{I,1}}\int_0^t (C_{B,s} - C_B(t))\,dt + T_{c,s} \tag{10}$$

$$F_{in}(t) = K_{c,2}\,(V_s - V(t)) + \frac{K_{c,2}}{\tau_{I,2}}\int_0^t (V_s - V(t))\,dt + F_{in,s} \tag{11}$$

Se usa el subíndice 1 para referirse a la ganancia del controlador de C_B y 2 para el de V.

2.2. Cálculo y elección del estado estacionario

En este subapartado, se pretende explicar el cálculo realizado para obtener los valores en estado estacionario, en los que se mantienen constantes a lo largo del tiempo,

y, las consideraciones oportunas para su elección, ya que T_c será un dato conocido dentro de un rango de valores previamente establecido (Bloor et al., 2025, p. 6) dando lugar a numerosos estados estacionarios, de los cuales se elegirá aquél en el que la productividad de B sea considerablemente alta, al ser el producto deseado, y propicie un buen funcionamiento de los controladores.

Primeramente, cabe tener en cuenta las ecuaciones disponibles para el cálculo. Los valores buscados no varían a lo largo del tiempo, por tanto, los términos de acumulación pertenecientes a los balances de materia mostrados en el apartado 2.1.2, $\frac{dx}{dt}$, son igual a 0.

Las ecuaciones quedarían de la siguiente manera:

$$0 = \frac{F_{in,s}\,(C_{A,in} - C_A)}{V_s} - r_a \tag{12}$$

$$0 = r_a - r_b - \frac{F_{in,s}\,C_{B,s}}{V_s} \tag{13}$$

$$0 = r_b - \frac{F_{in,s}\,C_{C,s}}{V_s} \tag{14}$$

$$0 = \frac{F_{in,s}\,(T_{in} - T_s)}{V_s} - \frac{\Delta H_a}{C_p\rho}\,r_a - \frac{\Delta H_b}{C_p\rho}\,r_b + \frac{UA\,(T_{c,s} - T_s)}{V_s C_p \rho} \tag{15}$$

$$0 = F_{in,s} - F_{out} \tag{16}$$

Además, los términos de las concentraciones presentes en las expresiones de las velocidades de reacción también representan las concentraciones en estado estacionario:

$$r_a = k_a\,\exp\left(-\frac{E_a}{RT}\right)\,C_{A,s} \tag{17}$$

$$r_b = k_b\,\exp\left(-\frac{E_b}{RT}\right)\,C_{B,s} \tag{18}$$

Todos los términos con el subíndice "s" representan el estado estacionario.

Con tal de resolver las ecuaciones, se considera que el volumen en estado estacionario (V_s) alcanza siempre los 100 m^3. Asimismo, la temperatura del fluido que circula por la camisa del reactor ($T_{c,s}$) recorre un rango de valores desde 290 hasta 450 K, variando de unidad en unidad. De este modo, se obtienen 160 estados estacionarios distintos, de los cuales se descartarán todos menos uno, de acuerdo con las pautas establecidas anteriormente.

Aunque pueda parecer que el punto a elegir debería ser aquél en el que la concentración

de B sea máxima, este no es el punto óptimo, ya que impide un buen funcionamiento del controlador.

Esto se debe a que, en el punto máximo, la variación de C_B respecto a T_c es igual a 0. Por tanto, en este caso concreto, el controlador, por mucho que modifique T_c, no podrá regular la concentración de B, dando lugar a un comportamiento no deseado e incluso inestable. Véase la siguiente figura 2:

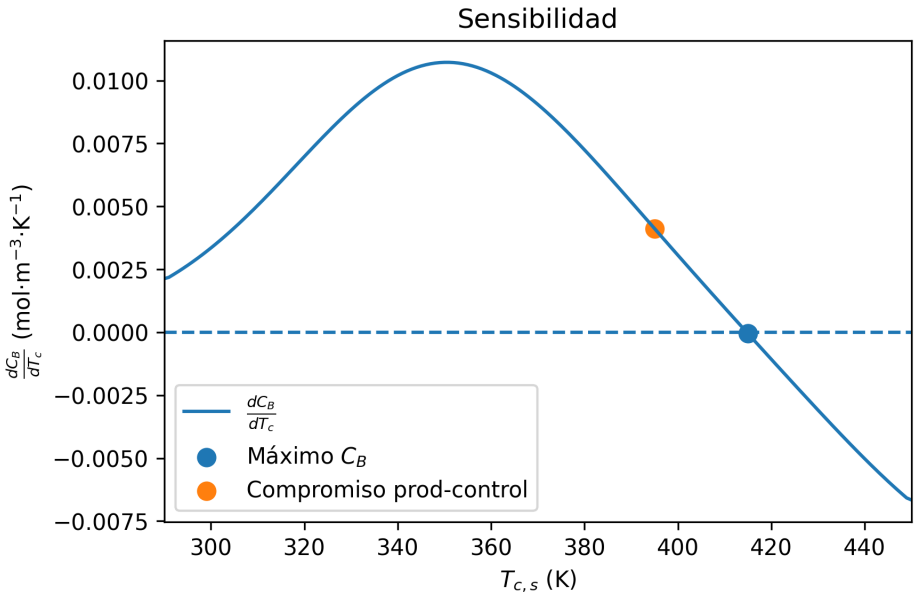


Figura 2: Sensibilidad de la concentración de C_B respecto a la temperatura de la camisa $T_{c,s}$.

Como solución, se ha optado por elegir el 95 % del valor máximo, con el objetivo de evitar el problema mencionado. En la gráfica se aprecia este valor representado mediante el punto naranja.

El valor de $T_{c,s}$ elegido, es igual a 395 K. El resto de valores corresponden a los calculados cuando $T_{c,s}$ adquiere dicho valor:

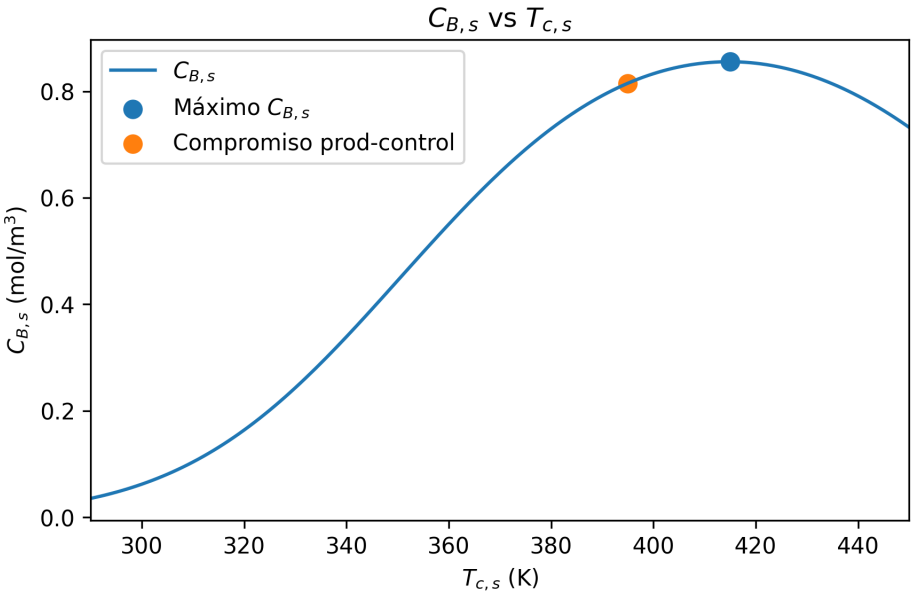


Figura 3: $C_{B,s}$ frente a $T_{c,s}$.

El valor de $C_{B,s}$ obtenido es 0,815 mol/m^3. En la Figura 3 se puede apreciar como, a medida que aumenta la temperatura de la camisa, aumenta también la concentración de B, hasta cierto punto.

Todos los valores en estado estacionario correspondientes a un $T_{c,s}$ de 395 K se recogen en la Tabla 2.

Tabla 2: Estado estacionario escogido

Parámetro	**Valor**	**Unidad**
$T_{c,s}$	395	K
$C_{B,s}$	0.815	$\mathrm{mol\,m^{-3}}$
$C_{A,s}$	0.162	$\mathrm{mol\,m^{-3}}$
$C_{C,s}$	0.023	$\mathrm{mol\,m^{-3}}$
V_s	100	$\mathrm{m^3}$
T_s	374.65	K
$F_{in,s}$	100	$\mathrm{m^3\,s^{-1}}$

Estos valores serán los utilizados en este caso de estudio.

2.3. Comportamiento del reactor

La intención de este apartado es estudiar cómo actúa el reactor y qué comportamientos son razonables para el RCTA del presente texto, con la finalidad de un buen entendi-

miento del mismo y avanzar poca a poco hasta la construcción de la red neuronal.

Es importante realizar las simulaciones pertinentes para comprobar si las EDOs que lo definen son coherentes, y, para poder detectar futuros defectos. Por ejemplo, en el muestreo realizado, para conocer datos que ocupen, no solo un amplio rango, sino también que tengan sentido físico, y, por tanto, tras las simulaciones, tener la capacidad crítica para que todos aquellos valores que disten de lo que cabría esperar, o no describan con verosimilitud el funcionamiento del RCTA, se descarten.

Todavía no se estudia el problema al completo, pues falta la implementación del controlador PI. El RCTA con PI será el modelo definitivo del cual se extraerán los datos para el entrenamiento de la red neuronal. Ésto será objeto del apartado 3.

Este apartado se desarrolla en base a las siguientes cuestiones: Cómo evolucionan las concentraciones de los componentes con el tiempo, y cuál es el efecto de la temperatura interna del reactor y del fluido en la camisa. No son preguntas que den lugar a un estudio exhaustivo, pues no es necesario, pero sí necesarias para cumplir con el objetivo de este apartado: Entendimiento del funcionamiento del RCTA y formación de razonamiento crítico.

2.3.1. Evolución de las concentraciones

Para conocer la tendencia que siguen las concentraciones, se considera que las variables manipuladas adquieren valores cercanos a los del estado estacionario previamente calculado. Esto es necesario, ya que aún no se ha implementado el controlador PI que permita regular dichas variables hasta que el resto de variables alcancen valores constantes y se puedan realizar las simulaciones.

Por ejemplo, la simulación se lleva a cabo con $T_c = 390$ K y $F_{in} = 140$ m^3 s^{-1}. Cabe recordar que las simulaciones no son más que las soluciones del sistema de ecuaciones diferenciales ordinarias (EDOs).

En este sistema, es necesario definir cuáles son las variables dependientes y cuál es la variable independiente. Las variables dependientes son: C_A, C_B, C_C, V y T. La variable independiente es el tiempo, expresado en segundos (t).

Resolviendo el sistema en Python mediante el método de Runge–Kutta, para un intervalo de tiempo de 25 segundos, se obtiene lo mostrado en la siguiente figura:

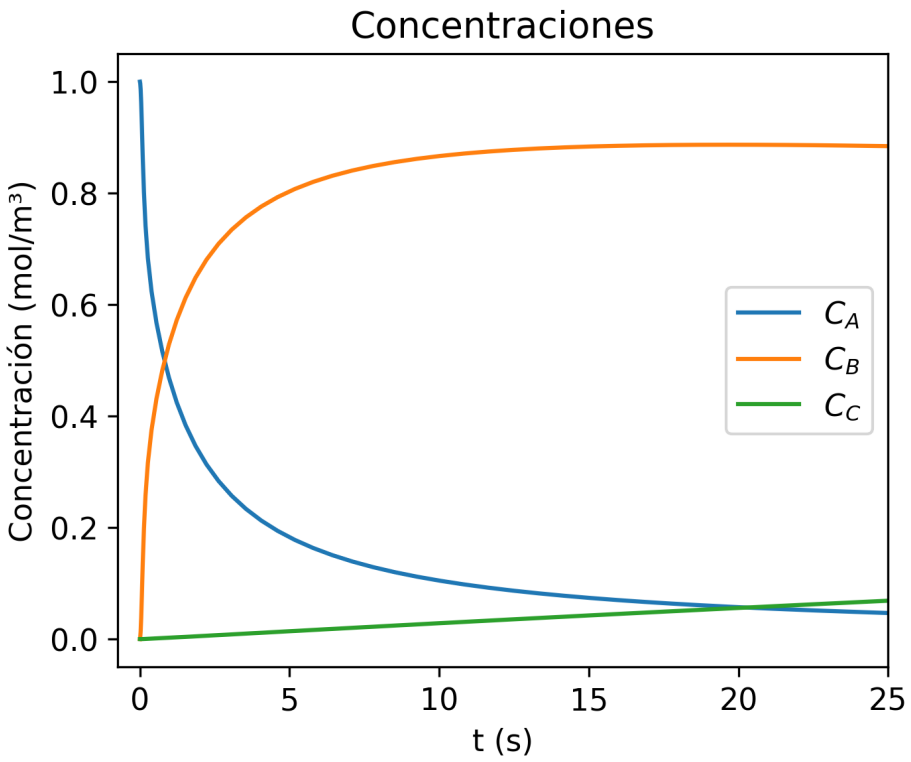


Figura 4: Evolución de las concentraciones.

La concentración de B aumenta, mientras A se consume disminuyendo su concentración. Ambas en un rango de 25 segundos están cerca de estabilizarse manteniendo sus valores constantes. Además, el compuesto C, que es un subproducto no deseado, aumenta muy poco su concentración en el tiempo. Son resultados esperables y con sentido físico.

2.3.2. Efecto de la temperatura

A partir de la simulación anterior se puede obtener también la evolución de la temperatura a lo largo del tiempo:

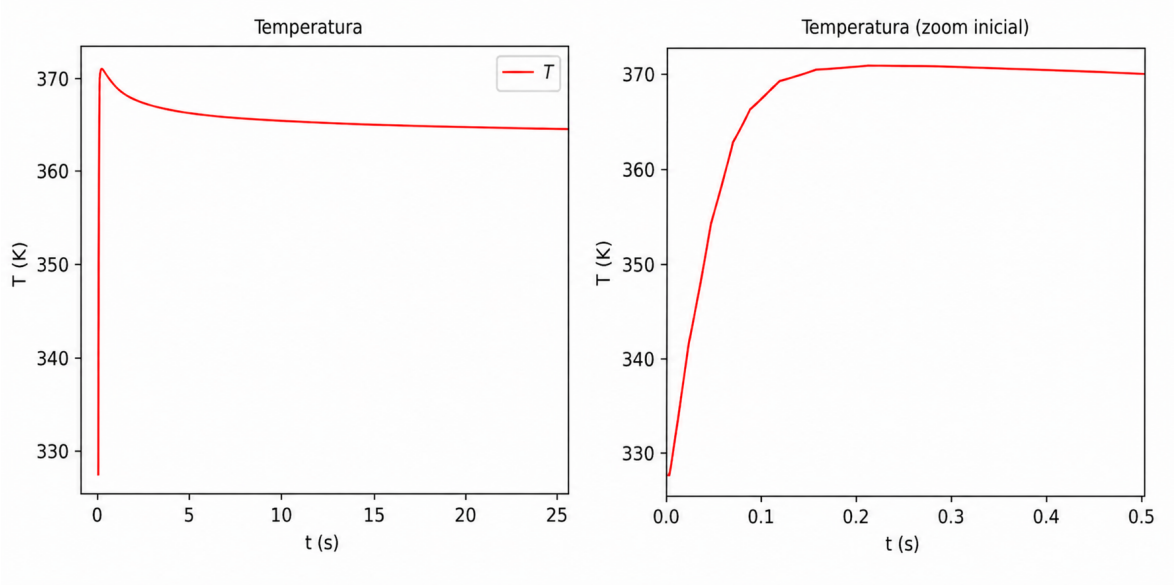


Figura 5: Evolución de la temperatura.

La ecuación (6) es la que proporciona este resultado. La temperatura presenta una subida repentina en un intervalo de tiempo muy corto desde su valor inicial (T_{in}), alcanzando aproximadamente 370 K, donde comienza a estabilizarse gracias al intercambio de calor con el fluido de la camisa.

Al reducir el eje horizontal a tan solo 0,5 s, se puede apreciar con mayor claridad la tendencia que sigue la temperatura en el inicio del proceso.

Resulta de interés analizar a qué se debe este rápido ascenso de T, ya que puede indicar qué variables presentan mayor inestabilidad o pueden generar mayores dificultades una vez se incorpore el controlador.

Para ello, se estudia el balance de energía (ecuación (6)) término a término, representando cada uno de ellos en función del tiempo:

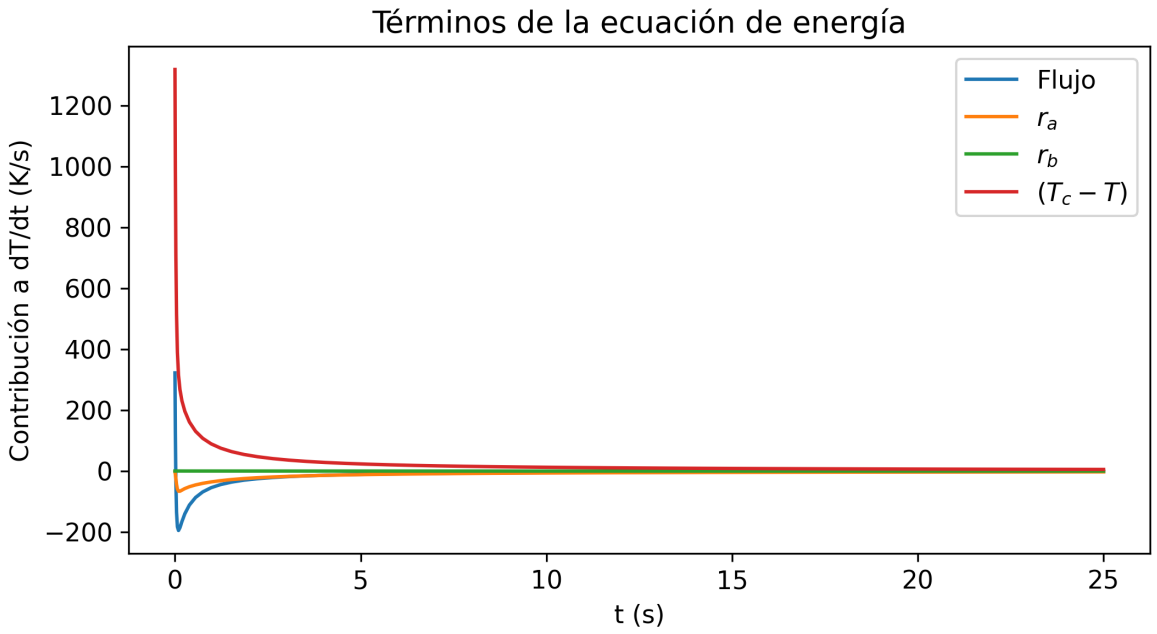


Figura 6: Contribución de cada término al balance de energía.

Se puede observar cómo el término de intercambio de calor contribuye de manera significativa al balance de energía en comparación con el resto de términos al inicio del proceso. Esto indica que el valor de T_c empleado (390 K) provoca una diferencia de temperaturas elevada.

Podría reducirse dicha contribución disminuyendo el valor de T_c, dado que se trata de un parámetro de prueba. Sin embargo, no resulta conveniente, ya que se busca mantener la mayor similitud posible con el comportamiento final del sistema una vez implementado el controlador PI, en el que se emplearán los valores en estado estacionario (Tabla 2). Por este motivo, se mantiene $T_c = 390$ K con el fin de analizar su influencia en la dinámica del sistema.

Para finalizar este apartado, queda determinar si el fluido de la camisa actúa como calefactor o como refrigerante. Para ello, se elimina el término de intercambio de calor del balance de energía, obteniéndose la siguiente expresión:

$$\frac{dT}{dt} = \frac{F_{in}\,(T_{in} - T)}{V} - \frac{\Delta H_a}{C_p \rho}\, r_a - \frac{\Delta H_b}{C_p \rho}\, r_b \tag{19}$$

Y se resuelve el sistema de ecuaciones diferenciales ordinarias (EDOs), ahora empleando la ecuación (19), para visualizar la evolución de T:

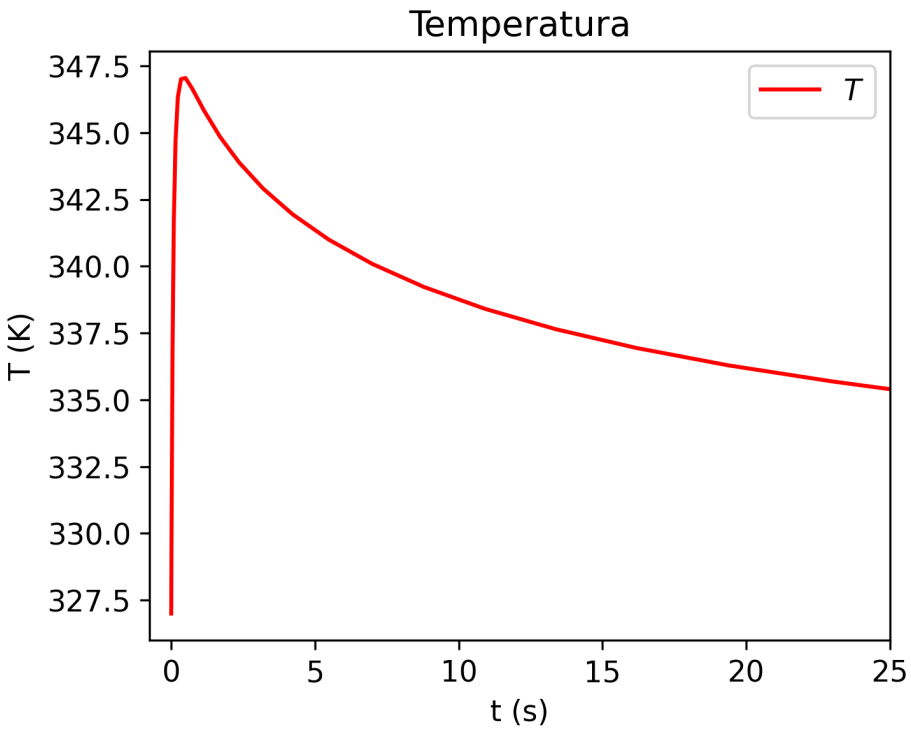


Figura 7: Evolución de T sin intercambio de energía.

Con esta comprobación, se demuestra que la reacción de A para dar lugar a B (la reacción de B para formar C apenas tiene lugar, tal y como se ha planteado) es endotérmica. Esto implica que la reacción extrae energía del sistema para poder llevarse a cabo, provocando una disminución de la temperatura T.

Como consecuencia, si T disminuye, es necesario emplear un fluido calefactor en la camisa del reactor que compense dicha pérdida de energía y permita mantener la temperatura estable.

Finalmente, se puede concluir que el fluido de la camisa actúa como calefactor, siendo T_c su temperatura.

No obstante, es necesario realizar una última aclaración respondiendo a la siguiente cuestión: si el término de intercambio de calor era el responsable del pico inicial de temperatura, ¿por qué, al eliminarlo del balance de energía, sigue observándose un aumento brusco de la temperatura, como se aprecia en la Figura 7. La respuesta se muestra en la figura 8:

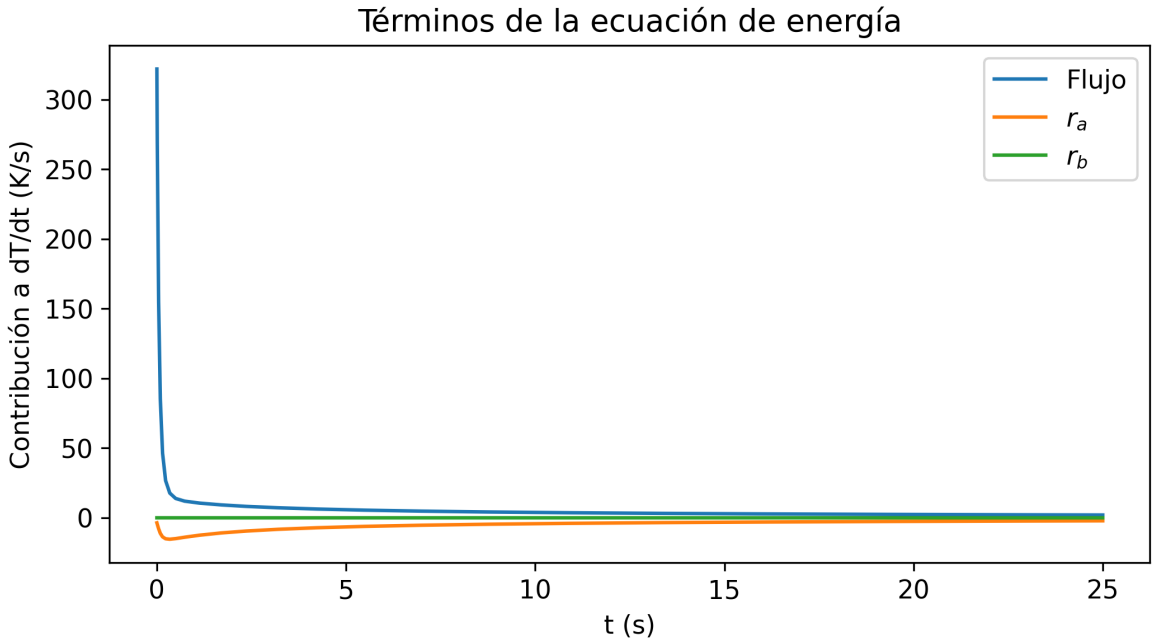


Figura 8: Evolución de los términos del balance de energía sin intercambio de calor.

Al volver a representar término a término la ecuación 19, ahora es el término del flujo a la entrada (F_{in}) el que contribuye en mayor medida que los otros dos, de manera que vuelve a aparecer una súbita subida de T al inicio del proceso.

2.4. Comportamiento del reactor con controlador PI

Como se ha comentado, se pretende que el reactor alcance los valores en estado estacionario calculados en el apartado 2.2. Para ello, se implementan 2 controladores PI, uno para C_B y otro para V.

Primeramente, cabe tener presente los parámetros de ambos controladores (Bloor et al., 2025, p. 8, Tabla 4):

Tabla 3: Parámetros del controlador

Ganancia	**Valor**	**Unidad**
$K_{c,1}$	3.09	$\mathrm{K\,m^3\,mol^{-1}}$
$\tau_{I,1}$	0.03	s
$K_{c,2}$	0.84	$\mathrm{s^{-1}}$
$\tau_{I,2}$	1.85	s

Estas ganancias serán las que se modificarán para realizar un muestreo de datos, como se verá posteriormente en el apartado 3.

Para implementar las acciones de ambos controladores en el sistema de ecuaciones diferenciales ordinarias (EDOs), se añaden dos ecuaciones adicionales, procedentes de las expresiones de cada controlador PI (ecuaciones (10) y (11)). De nuevo, se emplea Python para la resolución del sistema mediante la función `solve_ivp`.

La forma en la que se ha abordado la resolución problema, consiste en realizar un cambio de variable en la parte integral de la ecuación del PI. De este modo, a partir de la ecuación general (8), se obtiene:

$$\int_0^t \varepsilon(t)\, dt = I \tag{20}$$

A continuación, esta variable se expresa en forma diferencial, teniendo en cuenta que la derivada de la integral es su integrando:

$$\varepsilon(t) = \frac{dI}{dt} \tag{21}$$

Finalmente, agrupando las ecuaciones 9 y 21, se obtiene la expresión general empleada para la resolución del sistema:

$$c_s - c(t) = \frac{dI}{dt} \tag{22}$$

Aplicando este procedimiento a las ecuaciones (10) y (11), se obtienen las ecuaciones diferenciales que formarán parte del sistema de EDOs:

$$C_{B,s} - C_B(t) = \frac{dI_1}{dt} \tag{23}$$

$$V_s - V(t) = \frac{dI_2}{dt} \tag{24}$$

Una vez resuelto el sistema, se obtienen los valores de I_1 e I_2 a lo largo del tiempo, lo que permite calcular las variables manipuladas T_c y F_{in} a partir de las ecuaciones (10) y (11), actuando como controladoras:

$$T_c(t) = K_{c,1}\left(C_{B,s} - C_B(t)\right) + \frac{K_{c,1}}{\tau_{I,1}}\, I_1 + C_{B,s} \tag{25}$$

$$F_{in}(t) = K_{c,2}\left(V_s - V(t)\right) + \frac{K_{c,2}}{\tau_{I,2}}\, I_2 + F_{in,s} \tag{26}$$

En resumen, el sistema de EDOs está formado por las ecuaciones (3), (4), (5), (6), (7),

(23) y (24). Mediante las ecuaciones (25) y (26) se incorpora el efecto del controlador para alcanzar los estados estacionarios y obtener la evolución dinámica de las variables manipuladas (Apéndice C).

Véase a continuación el resultado de las concentraciones:

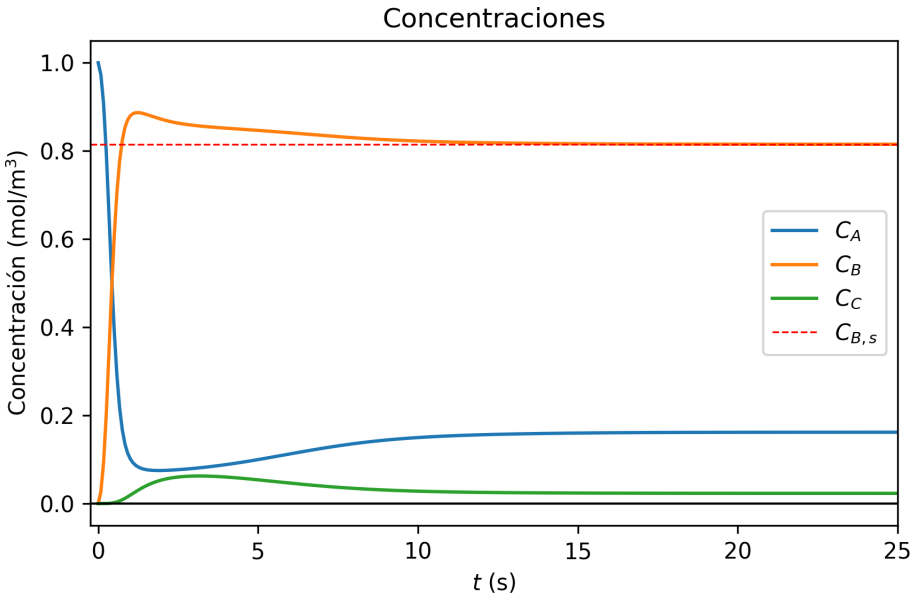


Figura 9: Control de las concentraciones.

C_B llega al estado estacionario calculado anteriormente (Tabla 2) al igual que el resto de concentraciones.

Ahora, el volumen:

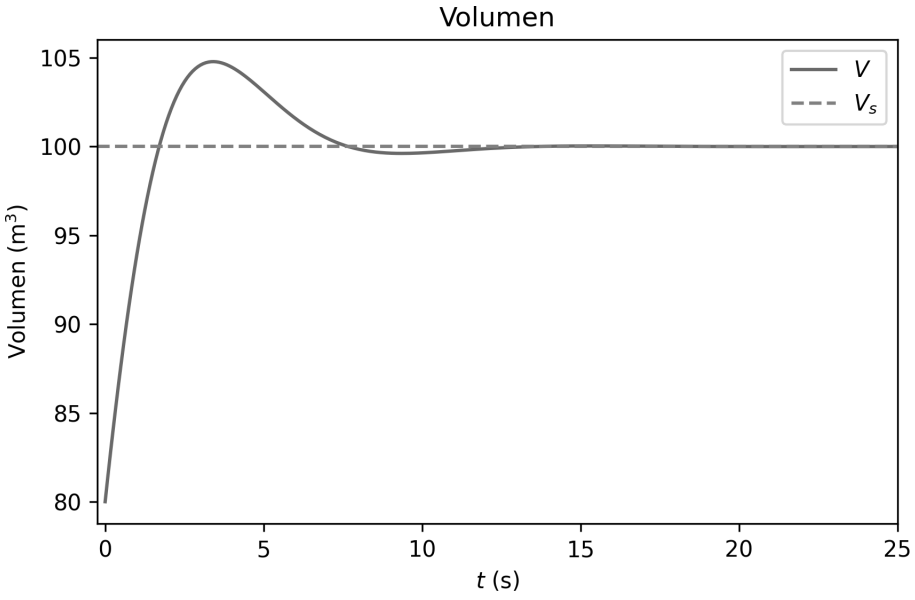


Figura 10: Control del volumen.

Al igual que las concentraciones, el resultado para el volumen es satisfactorio: se llega al estado estacionario deseado.

Compruébese también si la temperatura ha obtenido el resultado querido:

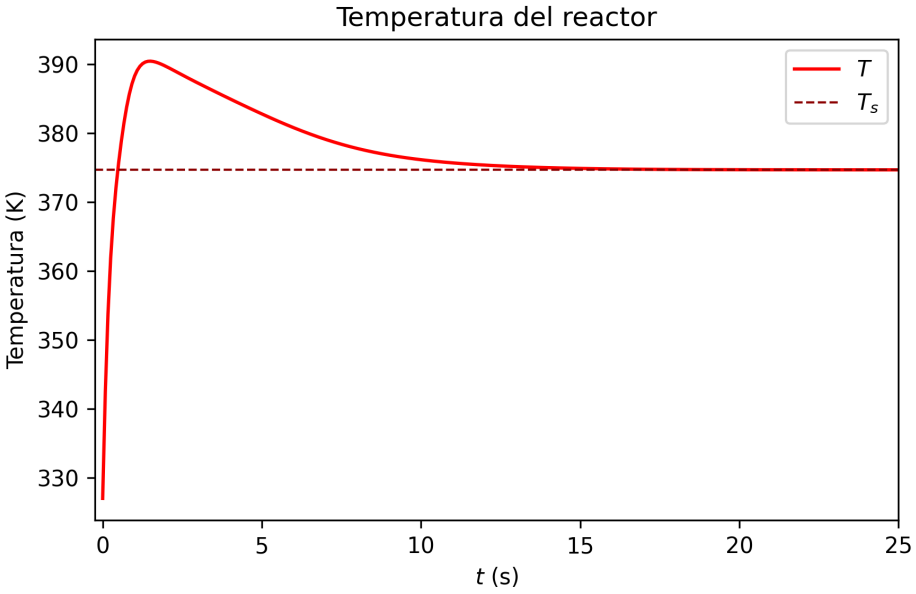


Figura 11: Control de la temperatura.

Visualícense ahora las variables manipuladas:

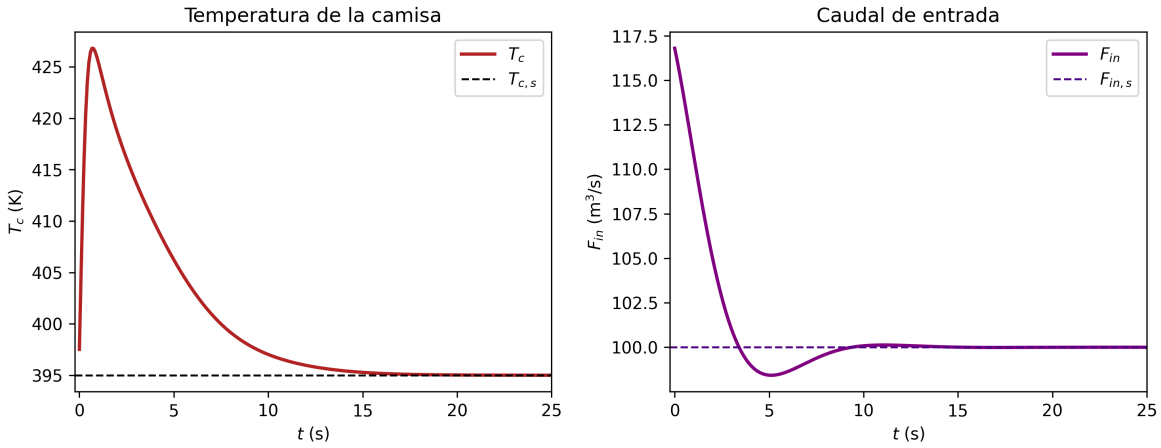


Figura 12: Control de las variable manipuladas.

Ambas alcanzan sus estados estacionarios correspondientes.

Por último, es interesante visualizar cómo evolucionan los errores:

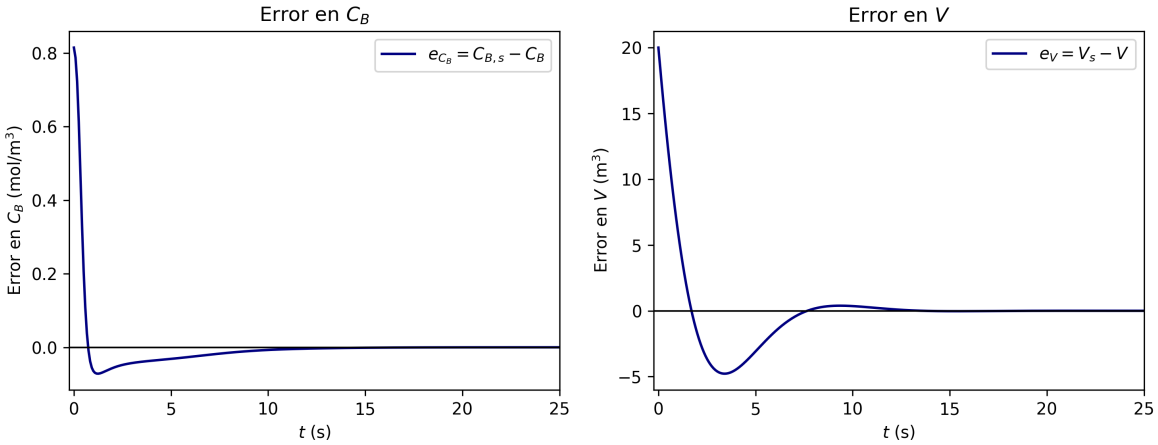


Figura 13: Control de las variable manipuladas.

3. OBTENCIÓN DE DATOS PARA EL ENTRENAMIENTO DE UNA RED NEURONAL.

Para que una red neuronal aprenda, es de vital importancia tener unos datos de buena calidad y bien estructurados, y para ello, se requiere de un trabajo de preprocesamiento de datos. En este caso de estudio, el preprocesamiento consta de varias etapas hasta llegar al documento final en el que están todos los datos recogidos (Dataset) y ordenados debidamente.

Primero, se generan datos por medio de un muestreo, en el que se cambian ciertos valores dentro de unos rangos, y así poder obtener distintos resultados para un mismo problema, en este caso, un RCTA con PI implementado. Luego, de esos datos, hay que descartar todos aquellos que no tengan sentido físico o no cumplan con los requisitos requeridos. Por último, hay que analizar los datos filtrados, para asegurarse de tener un Dataset que permita un buen aprendizaje de la red construída.

Es por ello, que en este apartado se explica el procedimiento realizado hasta la obtención del mencionado Dataset, y todas aquellas consideraciones tomadas para que sea de la mayor calidad posible.

3.1. Rango de los parámetros de los controladores PI.

Como se explicado en la introducción de este apartado, para muestrear el caso de estudio hay que cambiar unos parámetros dentro de unos rangos establecidos. Estos parámetros, serán los 4 existentes entre los 2 controladores PI. Los rangos de los parámetros se pueden ver en la siguiente tabla:

Tabla 4: Rangos de variación de los parámetros de los controladores PI

Parámetro	**Límite inferior**	**Límite superior**	**Unidad**
$K_{c,1}$	0.1	10	$\mathrm{K\,m^3\,mol^{-1}}$
$\tau_{I,1}$	0.01	5	s
$K_{c,2}$	0.1	10	$\mathrm{s^{-1}}$
$\tau_{I,2}$	0.01	5	s

3.2. Muestreo de datos

El muestreo se realiza con el código que se muestra en el Apéndice D.1., y es necesario entender qué método se está usando y cómo se recogen los valores.

Lo que hace exactamente este código, es elegir con la clase `.sobol` una combinación de los parámetros del controlador concreta dentro de los rangos establecidos.

La función `.sobol` se encuentra dentro de la biblioteca `Scipy`, concretamente en el módulo de muestreo quasi-Monte Carlo (`qmc`) del subpaquete `scipy.stats`.

Lo que hace esta función, es crear puntos de baja discrepancia, es decir, puntos distribuidos de manera que el espacio de diseño se cubra de forma más uniforme que con un muestreo aleatorio convencional, reduciendo agrupamientos y mejorando el espacio cubierto. De esta forma, los parámetros del controlador ocuparán los rangos establecidos ampliamente para recoger la mayor cantidad de combinaciones numéricas existentes posibles entre los 4 parámetros.

En la línea 85 se encuentra esta función:

```
85          sampler = qmc.Sobol(d=4, scramble=True)
```

Donde el primer argumento indica el número de parámetros a muestrear y el segundo aplica una aleatorización controlada.

Una vez elegida la combinación, se resueve el sistema de EDOs, en este caso por el método `BDF`, simulando el reactor. Esta simulación, se ha llevado a cabo en un rango de 0 a 40 segundos y está distribuída en 150 instantes temporales, dando lugar a 150 puntos de cada variable dentro de ese rango. Esto está expresado en la línea 58 por medio del siguiente código:

```
58          t0 = 0.0
59          tf = 40.0
60          t_eval = np.linspace(t0, tf, 150)
```

Esto se menciona porque podemos variar el número de datos dentro del Dataset en

función de en cuántos puntos se divida el tiempo, y puede llegar a ser útil, si es menester aumentar o dismunuir el número de datos. Con 150 puntos (para cada variable) se han logrado resultados satisfacotrios.

La función que resuelve el sistema de EDOs, ahora pasa a tener la siguiente forma:

```
90          def CSTR_PI(t, y, Kc_Cb, tauI_Cb, Kc_Fin, tauI_Fin):
91
92              Cb, T, V, Ca, Cc, I_Cb, I_V = y
```

Esto es debido a que, aunque la combinación de parámetros dada por la función `.Sobol` sea constante durante la resolución del sistema de EDOs, será distinta para cada simulación, pues dicha función los cambia cada vez que una simulación ha sido completada, obteniendo así simulaciones con dinámicas distintas.

El número de simulaciones se concreta justo antes de llamar a la función `.Sobol`:

```
82          m = 11
83          n_samples = 2**m
84
85          sampler = qmc.Sobol(d=4, scramble=True)
```

Dando lugar a 2048 simulaciones.

De manera compacta, 1 simulación, se obtiene de 1 combinación de parámetros del controlador elegida con `.Sobol()`, y cada simulación, se distribuye en 150 instantes temporales, formando una muestra dentro del Dataset. Luego, las muestras filtradas, serán las usadas en el entrenamiento y prueba de la red.

3.3. Filtrado de simulaciones dentro del Dataset

Al hacer el muestreo, se crean muchas muestras que se almacenan en el Dataset y no son apropiadas para que la red neuronal, aprenda las dinámicas en función de las ganancias de los controladores. Es por ello, que todas aquellas simulaciones que no lleguen al estado estacionario y no se mantengan estables durante 3 segundos al llegar a él, serán descartadas.

Estos son los dos condicionantes impuestos en el código creado para la filtración, expuesto en el Apéndice D.3.

Tras el muestreo, se obtenían los siguientes resultados:

```
1           Simulaciones correctas: 2048
2           Simulaciones fallidas: 0
3           Filas totales: 307200
```

Mientras que al descartar todas aquellas muestras que no presten los dos requisitos expuestos, se reduce más o menos a la mitad el tamaño del Dataset:

```
Simulaciones validas guardadas en dataset: 1040
Simulaciones descartadas: 1008
Filas totales: 156000
```

Un número considerable de simulaciones no alcanzaban el estado estacionario, eso quiere decir que, incluso dentro de un rango reducido, no todas las combinaciones de las ganancias de los PIs son válidas para el control del sistema.

Ya solo queda un útlimo paso antes de conseguir un Dataset bien estructurado para entrenar la red neuronal: su análisis. Pero antes, cabe hacer un inciso en el impacto que tiene la estructura de los datos en el entrenamiento de redes neuronales.

3.4. Estructura del Dataset.

Es realmente importante saber cómo es el Dataset, después de haber sido apilado en un solo documento y filtrado, ya que dependiendo de su estructura, la red aprenderá una cosa u otra.

Al ejecutar el código completo del Apéndice D.1., el Dataset tiene la siguiente forma:

Tabla 5: Estructura esquemática del conjunto de datos

Nº de filas	**sim_id**	t	**x**	**y**	**z**	**A**	**B**	**C**
1	1	0	x_1	y_1	z_1	cte1	cte2	cte3
2	1	1	x_2	y_2	z_2	cte1	cte2	cte3
3	1	2	x_3	y_3	z_3	cte1	cte2	cte3
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
q	1	40	x_q	y_q	z_q	cte1	cte2	cte3
$q+1$	2	0	x_{q+1}	y_{q+1}	z_{q+1}	cte4	cte5	cte6
$q+2$	2	1	x_{q+2}	y_{q+2}	z_{q+2}	cte4	cte5	cte6
$q+3$	2	2	x_{q+3}	y_{q+3}	z_{q+3}	cte4	cte5	cte6

Se presenta en forma de esquema para que sea legible y se pueda entender de mejor manera, ya que el Dataset real es mounstruoso y dificultaría el objetivo de este apartado, que es justificar por qué se ha elegido una determinada estructura del Dataset.

En la Tabla 5, x, y, z serían las variables dependientes del sistema como las concentraciones o temperaturas. A, B, C representan los parámetros de los controladores, constantes

en cada simulación y distintos entre ellas. Cada simulación, viene identificada debajo de la columna sim_id.

En este caso, cada fila es un tiempo concreto, donde se encuentran los valores correspondientes a cada variable para ese instante de tiempo.

No es recomendable esta estructura, ya que, la red neuronal aprenderá qué pasa en un instante concreto. Es decir, para todas las simulaciones, en el segundo 1, por ejemplo, intentará encontrar una relación, para en un futuro hacer una predicción precisa de cómo se comporta el reactor en el segundo 1, dependiendo de las ganancias del controlador. Podría ser posible, pero se necesitarían muchísimos más datos de los que ya hay, dificultando el aprendizaje.

Aún así, el autor de este texto lo intentó con esta forma de Dataset, pero los resultados fueron pésimos, requiriendo una gran cantidad de epochs (llegando a 2 horas de entrenamiento) para predecir resultados muy alejados de lo que se esperaría.

Se propone entonces hacer el siguiente cambio, en el que en cada fila, se guarda la simulación completa. Véase en el siguiente esquema:

Tabla 6: Estructura esquemática del conjunto de datos en formato ancho

`sim_id`	**A**	**B**	**C**	$x(0)$	$x(1)$	$\cdots$	$x(149)$	$y(0)$	$y(1)$	$\cdots$	$y(149)$
1	cte1	cte2	cte3	$x_1(0)$	$x_1(1)$	$\cdots$	$x_1(149)$	$y_1(0)$	$y_1(1)$	$\cdots$	$y_1(149)$
2	cte4	cte5	cte6	$x_2(0)$	$x_2(1)$	$\cdots$	$x_2(149)$	$y_2(0)$	$y_2(1)$	$\cdots$	$y_2(149)$
3	cte7	cte8	cte9	$x_3(0)$	$x_3(1)$	$\cdots$	$x_3(149)$	$y_3(0)$	$y_3(1)$	$\cdots$	$y_3(149)$

El código para reformular el Dataset puede encontrarse en el Apéndice D.4. y se ejecutará al Dataset filtrado.

Con esta estructura, lo que aprende la red neuronal es qué dinámica debe tener cada variable según su combinación de las ganancias del controlador correspondiente.

Así, no relaciona un valor de una variable a un tiempo, sino, una variación completa, a los parámetros de ambos PIs.

Dicho esto, se presenta el análsis de datos realizado.

3.5. Análisis de las muestras filtradas.

Anteriormente, se ha introducido que del muestreo realizado se deben obtener datos de calidad. Para la red neuonal que se quiere presentar y el tipo de problema que se ha desarrollado, para que haya cierta calidad debe cumplirse que haya variación entre las simulaciones en los primeros segundos del proceso, dando lugar a varias dinámicas y que todas ellas lleguen al estado estacionario deseado, o cerca de él.

Esto es fundamental para que la red neuronal aprenda, en lugar de memorizar una misma dinámica. Por ello, este subapartado tiene como objetivo comprobar de manera gráfica, principalmente, si realmente los datos filtrados son lo suficientemente distintos entre ellos como para que la red neuronal aprenda de la manera más precisa posible.

3.5.1. Espacio ocupado de las ganancias de los PIs.

Primeramente, hay que comprobar si el muestreo ha sido efectivo, y las ganancias ocupan todos los rangos mencionados en el apartado 3.1..

Para ello, se utliza el código mostrado en el Apéndice D.5., obteniendo los siguientes resultados.

Primero entre las variables, de un mismo controlador (1 y 2 por separado):

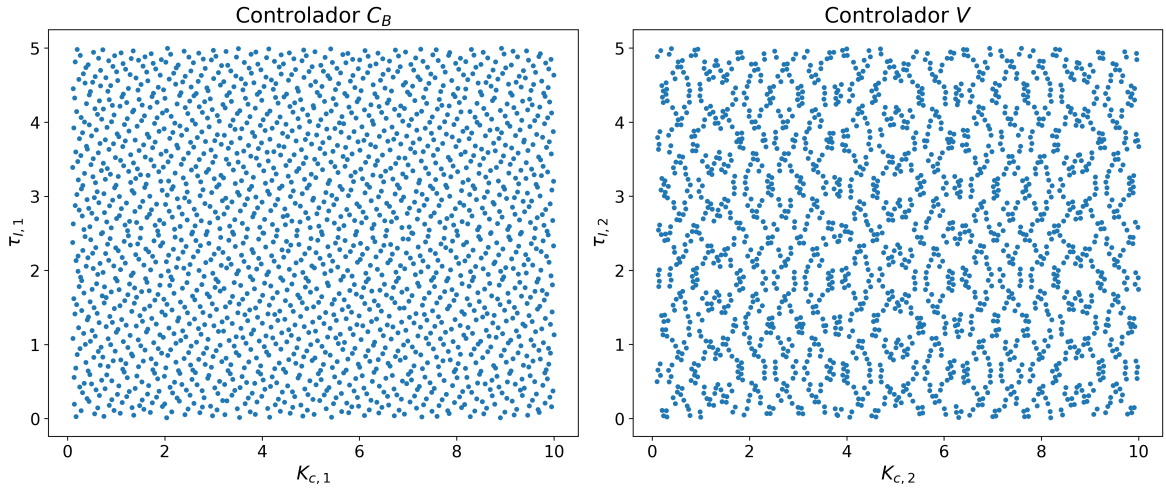


Figura 14: Distribución de los parámetros de los controladores generados mediante muestreo Sobol. A la derecha para controlador 1 y a la izquierda para el 2.

Y ahora, entre parámetros de controladores distintos:

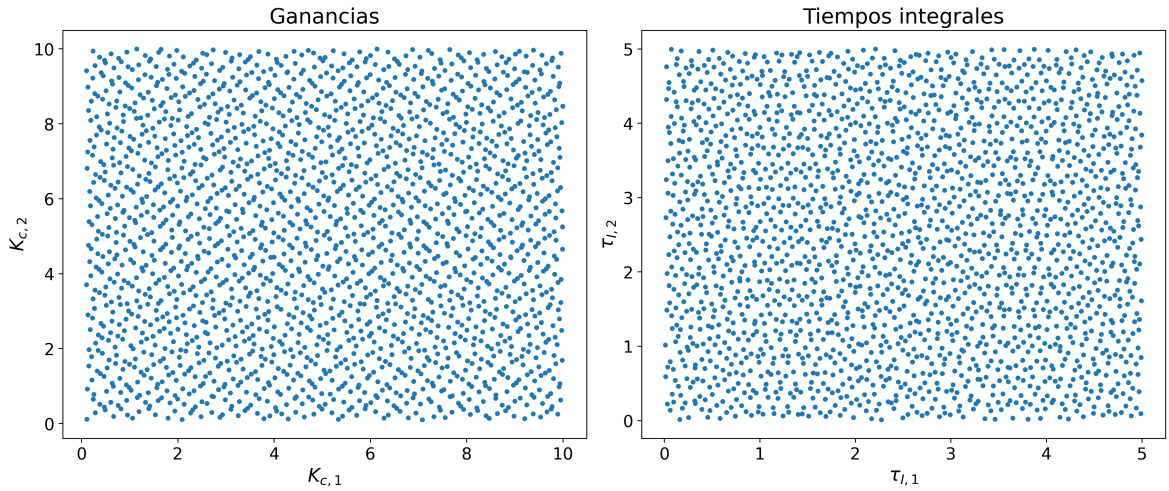


Figura 15: Distribución de los parámetros de los controladores generados mediante muestreo Sobol. A la derecha las ganancias y a la izquierda los tiempos integrales.

Se obtienen resultados muy satisfactorios, recorriendo todo el espacio de los rangos establecidos.

Además, para mayor profundidad, es posible también ver gráficamente qué valores de los parámetros de los PIs es más propenso a dar resultados que se almacenen en el Dataset de entrenamiento, distinguiendo entre simulaciones guardadas y desacartadas.

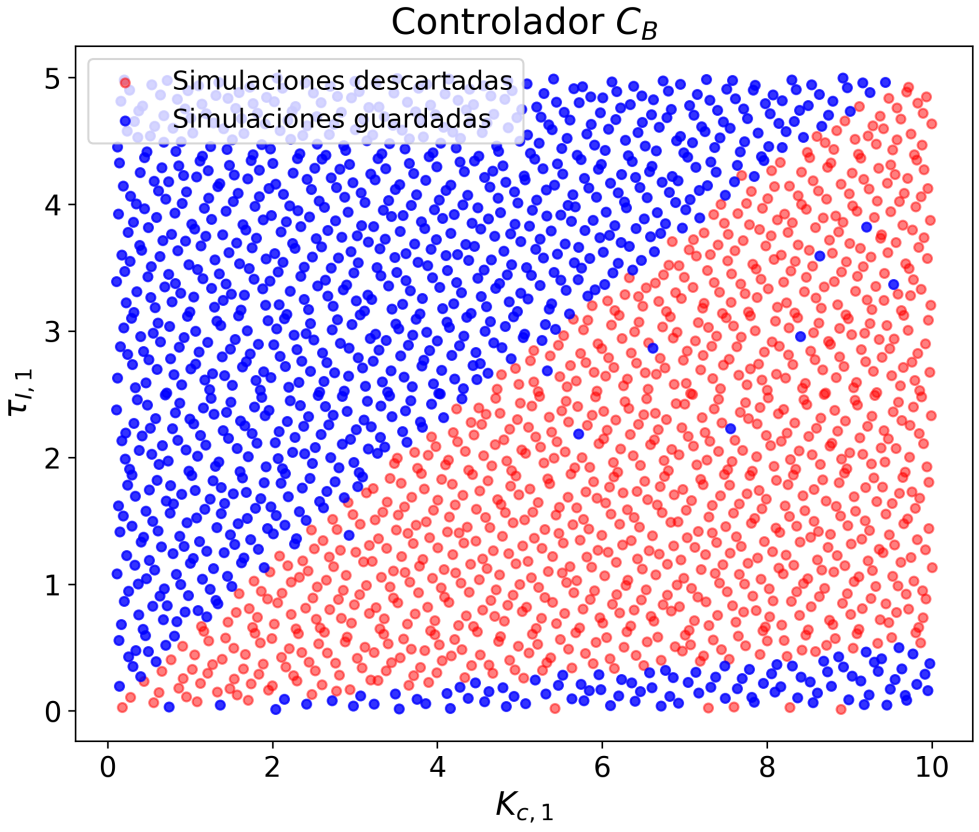


Figura 16: Comparativa de simulaciones guardadas y descartadas por medio de los parámetros del controlador 1.

Para el controlador 1 (o C_B), con valores de $K_{c,1}$ muy altos y $\tau_{I,1}$ muy bajos es muy poco probable que la simulación sea apta para el Dataset, habiendo una pequeña franja de simulaciones buenas en valores de $\tau_{I,1}$ muy cercanos a 0.

En cambio, cuando los valores de $\tau_{I,1}$ se van acercando a 5 y los de $K_{c,1}$ siguen siendo altos, la mayoría de simulaciones son satisfactorias. Esto es positivo para que la red aprenda todo tipo de combinaciones y sus dinámicas.

Para el controlador 2 (o V), cualquier combinación de parámetros puede ser descartada o deseada dentro de sus rangos:

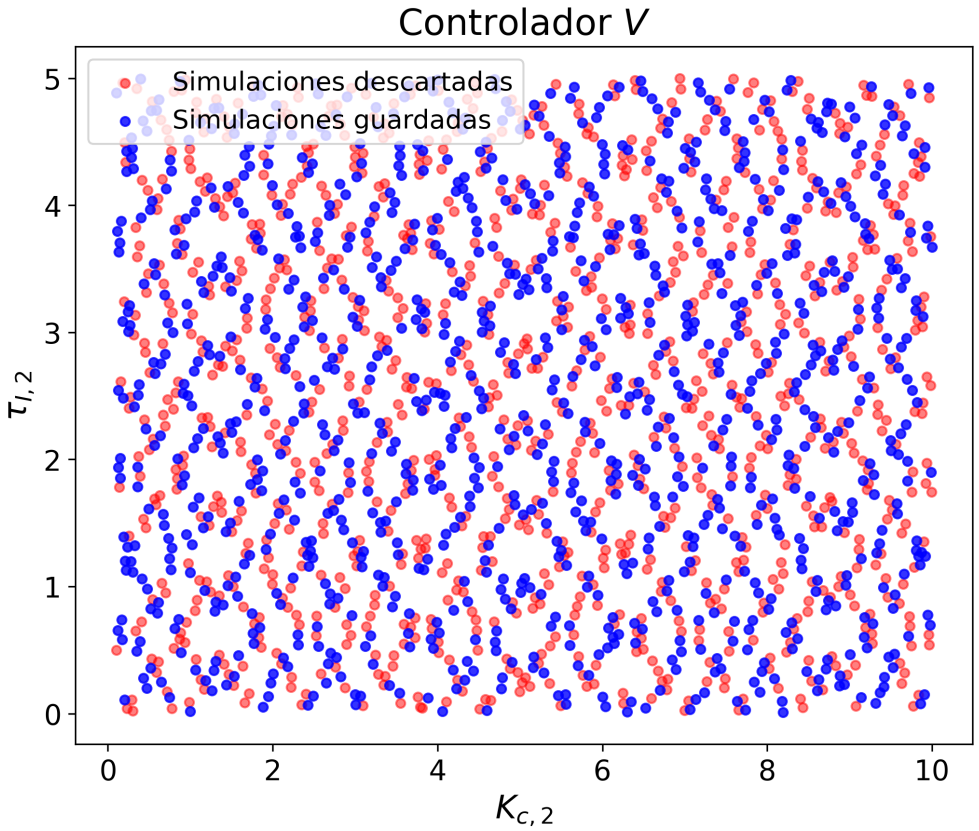


Figura 17: Comparativa de simulaciones guardadas y descartadas por medio de los parámetros del controlador 2.

Estas mismas conclusiones se pueden extraer por medio de las gráficas con los parámetros cruzados:

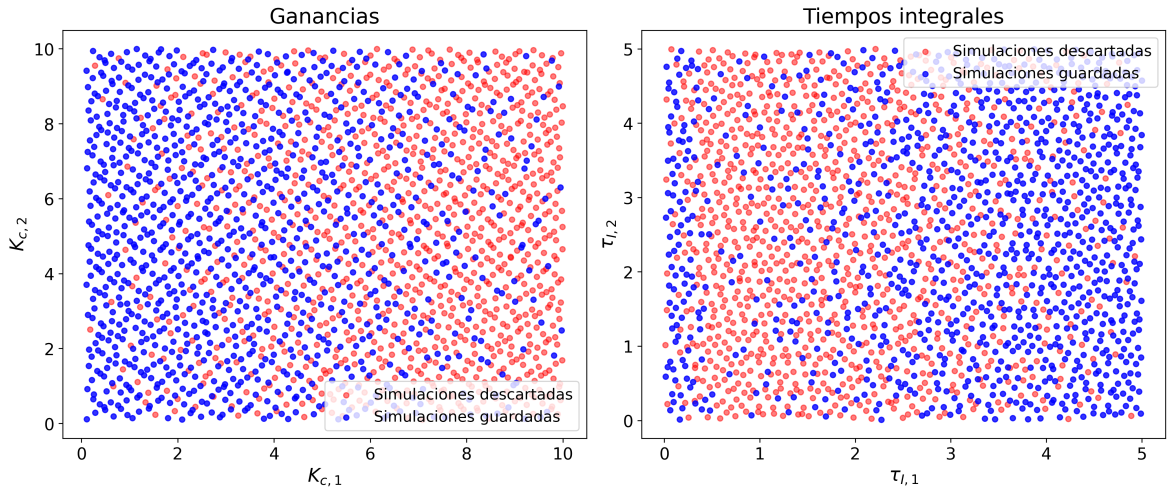


Figura 18: Comparativa de simulaciones guardadas y descartadas por medio de los parámetros cruzados.

3.5.2. Gráficos de variabilidad.

Una vez expuesto el espacio ocupado de los parámetros de los PIs, hay que comprobar además si realmente existe variabilidad entre las dinámicas de las variables. Principalmente, se estudiarán en este subapartado las dinámicas de C_B, T y V, las cuales son las que se desean predecir.

El código para lo que se presenta a continuación está, en el Apéndice D.6..

Los gráficos que se presentan a continuación, tienen como finalidad mostrar el área sobre la que las dinámicas de todas las simulaciones filtradas pueden encontrarse. Además, también se presenta otro gráfico, mostrando cuál es la desviación estándar (std) en función el tiempo, esperando que al principio haya una mayor variación, y se reduzca medida que se aproxime al estado estacionario.

La desviación estándar calculada, no es una desviación estándar temporal dentro de una sola simulación, sino una desviación estándar entre simulaciones en cada instante, calculándose de la siguiente forma:

$$\sigma(t_j) = \sqrt{\frac{1}{N}\sum_{i=1}^{N}\left(x_i(t_j) - \bar{x}(t_j)\right)^2} \tag{27}$$

$$\bar{x}(t_j) = \frac{1}{N}\sum_{i=1}^{N} x_i(t_j) \tag{28}$$

En donde:

- $\sigma(t_j)$: desviación estándar de la variable analizada en el instante de tiempo t_j.

- t_j: instante de tiempo correspondiente a la columna j de la matriz de datos.
- N: número total de simulaciones consideradas.
- $x_i(t_j)$: valor de la variable en la simulación i en el instante de tiempo t_j.
- $\bar{x}(t_j)$: valor medio de la variable en el instante de tiempo t_j, calculado como:

$$\bar{x}(t_j) = \frac{1}{N} \sum_{i=1}^{N} x_i(t_j)$$

- i: índice que identifica cada simulación, con $i = 1, 2, \ldots, N$.
- j: índice que identifica cada instante de tiempo.

En el código empleado, la variación estándar se calcula en la siguiente línea:

```
23          std_over_time = np.std(X, axis=0)
```

Variabilidad de C_B

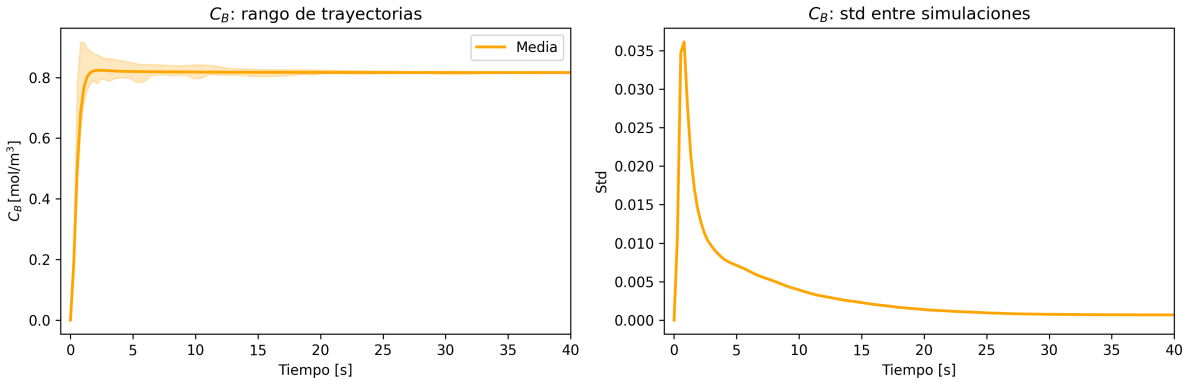


Figura 19: Rangos de simulaciones y std para C_B.

Esta variable es la menos sensible de todas al cambio de los parámetros de los PIs. En los primeros segundos de las simulaciones, sí existe una variación notable entre ellas, como se puede apreciar en la Figura 19.

En la siguiente, Figura 20, se muestran 5 simulaciones aleatorias de las 1040 guardadas en el Dataset final, con un zoom en la parte inicial, para poder distinguirlas mejor. Hay cierta diferencia en las dinámicas entre ellas, aunque no demasiada, suficiente para entrenar a la red neuronal, como se verá en futuros apartados de este texto:

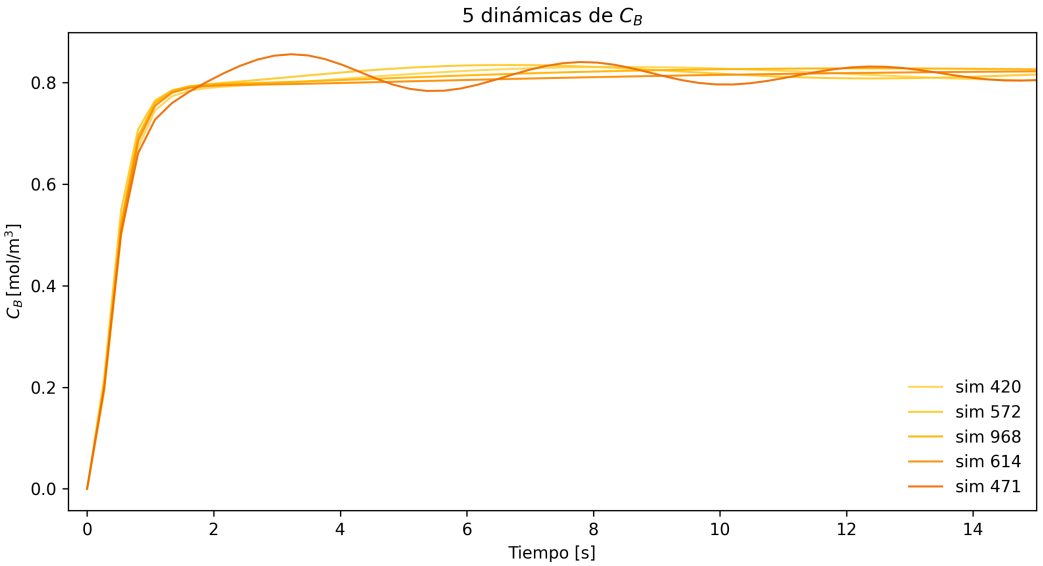


Figura 20: 5 simulaciones de C_B.

Variabilidad de T

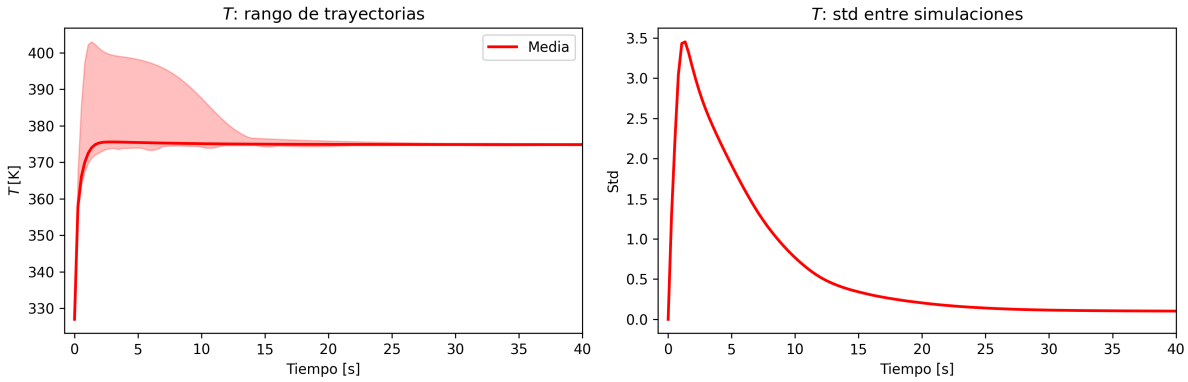


Figura 21: Rangos de simulaciones y std para T.

En esta ocasión, las dinámicas de T se dsitribuyen por un área bastante más amplia y apreciable que las de C_B, ofreciendo distintos comportamientos sobre los que la red neuronal aprenderá.

En la Figura 22 se muestran algunos ejemplos:

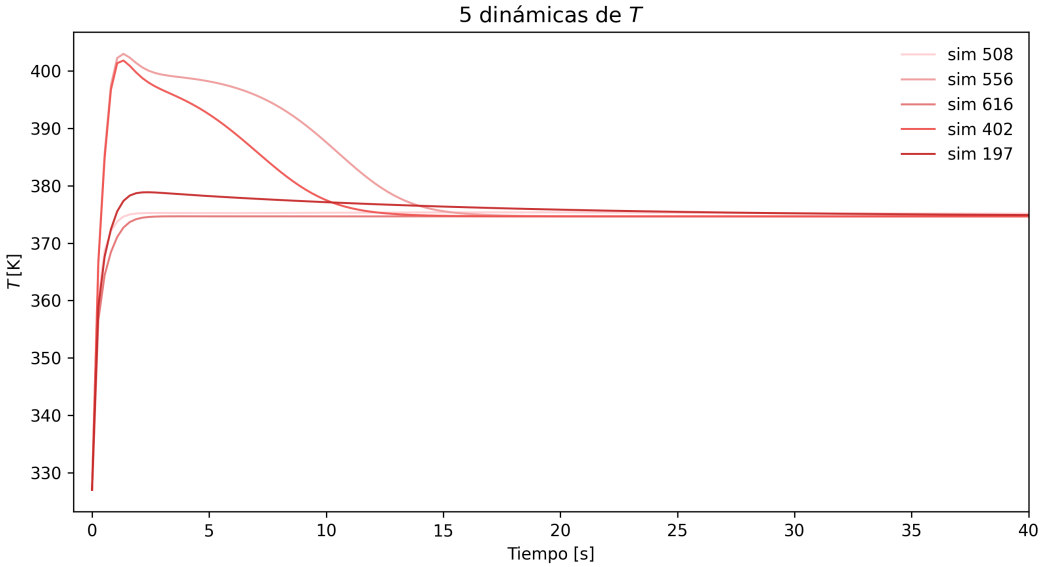


Figura 22: 5 simulaciones de T.

Variabilidad de V

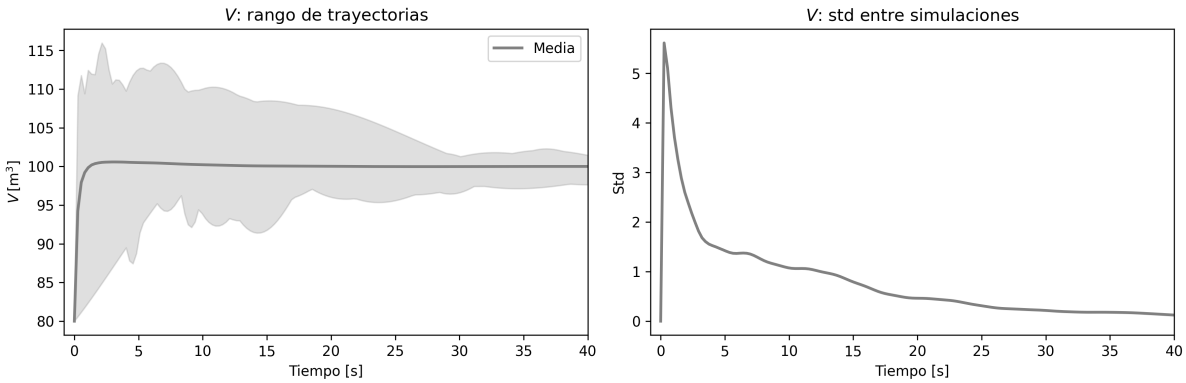


Figura 23: Rangos de simulaciones y std para V.

Sin duda alguna, V es la más sensible al muestreo realizado, aportando un área más amplia sobre la que se mueven las distintas dinámicas de V.

Como en los casos anteriores, se enseñan 5 ejemplos:

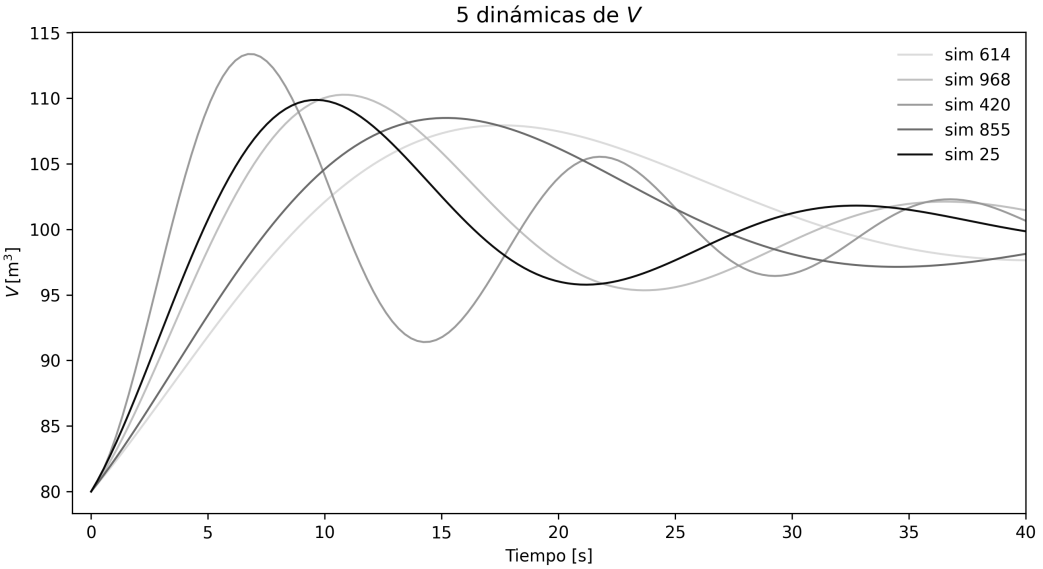


Figura 24: 5 simulaciones de V.

Finalmente, se conlcuye que el Dataset obtenido, puede ser fiable para el entrenamiento de la red, pues ofrece suficiente variabilidad, y, por tanto, distintos comportamientos sobre los que aprenderá, evitando así que memorice en lugar de que aprenda. Además, todas las simulaciones son fieles al caso de estudio, representando dinámicas realistas y propias del problema definido.

4. CONSTRUCCIÓN DE LA RED NEURONAL

Tras calcular los estados estacionarios deseados, entender cuál es el comportamiento del RCTA con PI estudiado, y obtener datos que representen distintos comportamientos del reactor en función de los parámetros de los controladores, se puede comenzar con la construcción de la red neuronal.

El objetivo de esta red neuronal, será predecir el comportamiento del reactor. Para ello, primero se deben separar los datos, de manera que cumplan dos funciones fundamentales. Una parte de ellos, y la más abundante, serán los de entrenamiento. La otra parte, serán datos de prueba para evaluar cómo se comporta la red neuronal con datos nuevos que aún no ha visto.

Se verá cómo se entrena, se evalúa y predice el modelo construído, explicando todo aquello que quepa tener en cuenta para su buen entendimiento.

Por último, se pondrán a prueba las predicciones que puede hacer, y se hará una valoración de sus limitaciones y posibilidades para mejorar la red neuronal.

Cabe recalcar que, el código empleado para dicho propósito se hará utilizando Tensorflow

+ Keras. Tensorflow es una librería de Python ampliamente utilizada para desarrollar modelos de Deep learning. A su vez, Keras es otra librería que facilita la contrucción y entrenamiento de redes neuronales. Todo el código empleado está expuesto en el Apéndice E.

4.1. Datos de entrenamiento y datos de prueba.

En el Apéndice E.2. se muestra el código empleado para hacer la separación de los datos.

Primero, se definen qué columnas serán las entradas:

```
7         input_cols = [
8         "Kc_Cb",          #Kc_1
9         "tauI_Cb",        #tauI_1
10        "Kc_Fin",         #Kc_2
11        "tauI_Fin"        #tauI_2
12        ]
```

Serán los parámetos del controlador. Luego, las salidas serán el resto de columnas que componen el Dataset (llamado dataset_wide):

```
20        output = dataset_wide.drop(columns=input_cols).values
```

De esta forma, aprende qué dinámica debe predecir, según qué valores tienen los parámetros mencionados.

Luego, con `train_test_split` se separan las entradas (input) y salidas (output) en datos de entrenamiento (train) y prueba (test). Los datos de prueba son el 25 % y los de entrenamiento, el 75 % del total:

```
26        input_train, input_test, output_train, output_test =
              train_test_split(
27        input,
28        output,
29        test_size=0.25,
30        random_state=42
31        )
```

4.2. Normalización

Al haber variables que oscilan en rangos suistancialmente distintos, es imperativo normalizar los valores para que se encuentren todos dentro del mismo intervalo y

facilitar el aprendizaje. En este caso, se normaliza de tal forma que recorran los valores de 0 a 1.

Para ello, se usa la siguiente función (Apéndice E.3.):

```
def norm_in/output(x):
return (x - train_stats_output['mean']) / train_stats_output['
    std']
```

Ahora, las variables con los datos normalizados pasan a tener la terminación `_norm`.

Para comprobar si han sido normalizados, al emplear el método `.describe()` debe verse cómo los valores de la columna `mean` (media) se aproximan a 0 y la desviación estándar adquiere un valor de 1. Aquí un ejemplo con los datos de entrada:

```
input_train_norm.describe().transpose()
```

Tabla 7: Estadísticas descriptivas de las variables de entrada normalizadas.

Variable	Número de datos	Media	Desviación estándar
$K_{c,1}$	884	$-5{,}22 \times 10^{-17}$	1.00
$\tau_{I,1}$	884	$1{,}95 \times 10^{-16}$	1.00
$K_{c,2}$	884	$-2{,}17 \times 10^{-16}$	1.00
$\tau_{I,2}$	884	$-1{,}77 \times 10^{-16}$	1.00

Por tanto, el código funciona correctamente y los datos están normalizados.

4.3. Definición del modelo.

La clase principal usada en Keras es `Sequential()`, que permite la creación de una red neuronal básica. y es la que se usará en este proyecto.

En este caso, el modelo de Keras se considera como una secuencia de capas, cada una de ellas, procesando gradualmente los datos de entrada para obtener la salida deseada.

La construcción en Keras del modelo, ha sido la mostrada en el Apéndice E.4. Ya que es la red neuronal en sí, punto neurálgico de este proyecto, cabe explicar brevemente qué se ha utilizado para su construcción.

Primeramente, la clase utilizada se llama en la línea 6 del código:

```
6 model = Sequential()
```

Luego, empiezan a crearse las capas de neuronas. En este proyecto, se han utilizado 3 capas: 2 capas ocultas y 1 capa de salida.

Las capas se añaden con `model.add()`, y en su argumento se indica el número de neuronas que tiene la capa, la función de activación, y un regularizador. Este ha sido el código utlizado para la primera capa oculta:

```
7         model.add(Dense(
8         128,
9         activation='relu',
10        kernel_regularizer=regularizers.l2(0.0001),
11        input_shape=(input_train_norm.shape[1],)
12        ))
```

`Dense` indica que todas las neuronas de una capa están conectadas con todas las neuronas de la siguiente capa, lo que se conoce como estar densamente conectadas. En la Figura 25 se aprecia de forma esquemática la estructura creada.

Tan solo para esta capa es necesario indicar el tamaño de la entrada a la red neuronal (`input_shape=()`). El número de neuronas empleado en esta capa es 128.

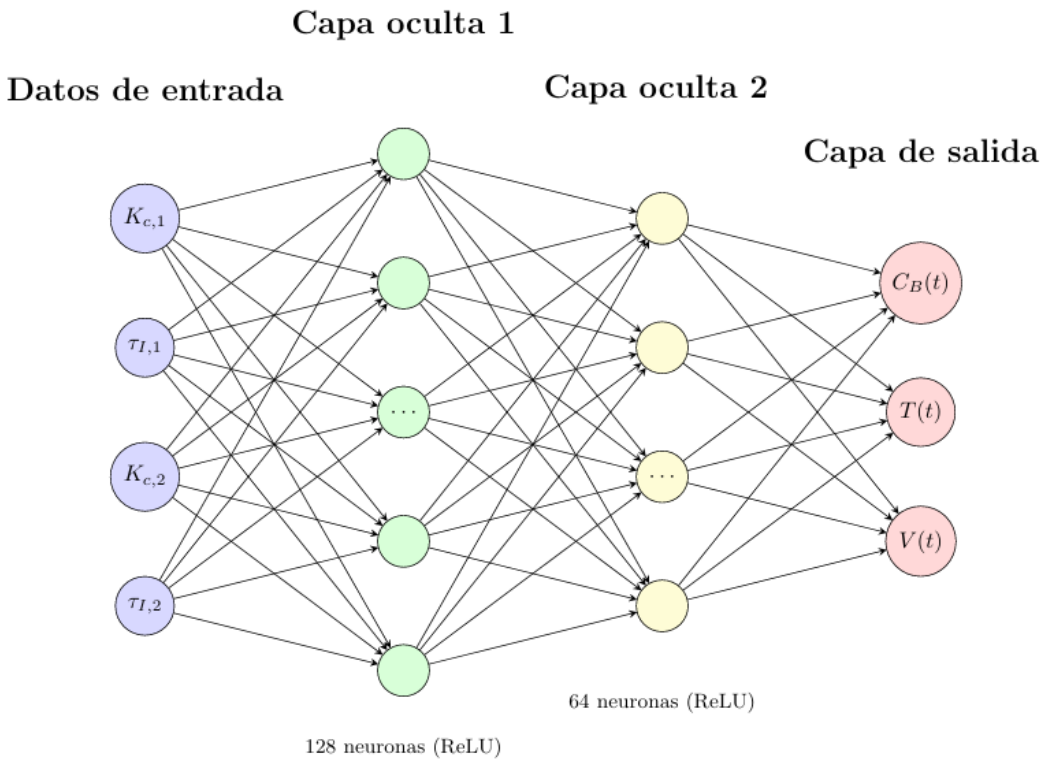


Figura 25: Esquema de la red neuronal.

En la línea 9, se indica qué función de activación se ha usado.

Brevemente, la función de activación aporta no linealidad a la red neuronal. Para este proyecto es clave, ya que debe aprender dinámicas claramente no lineales.

Durante el proceso de aprendizaje se ajustan parámetros: pesos y sesgos, que tienen una relación lineal con la salida de cada neurona. Esquemáticamente podría verse así:

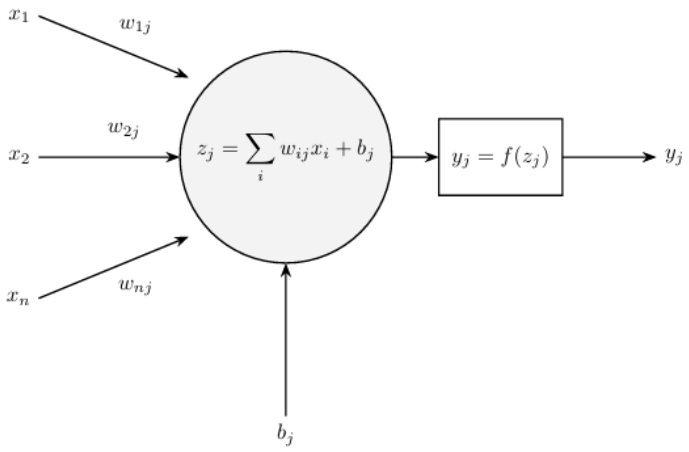


Figura 26: Esquema de cada neurona.

Siendo 'x' los datos de entrada, 'y' el valor que se intenta predecir, 'w' los pesos y 'b' el sesgo y 'z' la función lineal, que luego pasa por la función de activación para romper con su linealidad, siendo esto representado en el rectángulo.

El ajuste de pesos y sesgos se hará de tal forma que simulen las funciones propias del reactor expuestas en el apartado 2, sin necesidad de saber cuáles son estas funciones. Sin la función de activación, esto no sería prosible.

La función de activación empleada es la función ReLU (Rectified Linear Unit). Y consiste en la siguiente ecuación:

$$\text{ReLU}(x) = \begin{cases} x & \text{si } x > 0 \\ 0 & \text{si } x \leq 0 \end{cases}$$

En la línea 10 se añade un regularizador de pesos. La regularización, consiste en modular la cantidad de información que el modelo puede almacenar con el fin de mejorar la posibilidad de generalizar el modelo.

En este caso se ha empleado un regularizador L2, que penaliza aquellos pesos que sean demasiado grandes, multiplicando las funciones de pérdida por 0.0001, ayudando así a suavizar las dinámicas y evitar picos.

En la línea 15, se añade la siguiente capa pero esta vez con 64 neuronas.

Por último, en la línea 22, está la capa de salida:

```
22        model.add(Dense(output_train_norm.shape[1]))
```

Las salidas deben ser los valores de la concentración de B, la temperatura y el volumen en todos los instantes de tiempo. Según la estructura de datos, la forma (`.shape`) de

`output_train_norm` es (780, 1650). Al añadirle el argumento `[1]` se hace referencia al segundo valor del tuple creado.

1650 es el número de columnas de `output_train_norm`, y debe ser el número de neuronas de la última capa.

El número total de parámetros que la red debe ajustar se puede saber con la siguiente línea de código:

```
model.summary()
```

Y se obtiene:

Tabla 8: Resumen de la arquitectura de la red neuronal

Layer (type)	**Output Shape**	**Param #**
dense_9 (Dense)	(None, 128)	640
dense_10 (Dense)	(None, 64)	8,256
dense_11 (Dense)	(None, 1642)	106,730
Total params		115,626
Trainable params		115,626
Non-trainable params		0

La red debe ajustar un total de 115.626 parámetros.

4.4. Configuración del modelo.

Al configurar el modelo, se indica cómo debe ser el proceso de aprendizaje con el método `.compile()` con el que se pueden especificar algunas propiedades a través de argumentos del método.

El uso de este método se ve de la manera siguiente:

```
7  model.compile(
8        loss='mae',
9
10       optimizer=optimizer,
11
12       metrics=['mae']
13       )
```

El primer argumento, es la función de pérdida `loss` y se utiliza el hiperparámetro `'mae'` (mean absolute error):

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i| \tag{29}$$

Donde:

- y_i: valor real.
- $\hat{y}_i$: valor predicho.
- n: número de muestras.

La función de pérdida tiene el objetivo de medir cuán lejos está la salida de lo que se espera conseguir: Coge las predicciones de la red y el valor verdadero (lo que se espera) y calcula un error cometido en una muestra de entrada específica.

Luego, genera una señal que se retroalimenta par ajustar el valor de los parámetros en una dirección que disminuya el error, hasta acercarse lo máximo posible al valor real.

Este ajuste, es lo que realiza el optimizador, que es el segundo argumento llamado `optimizer` y el hiperparámetro es `learning_rate`, que se escribe a continuación:

```
4        optimizer = tf.keras.optimizers.Adam(learning_rate=0.0001)
```

Los optimizadores, ajustan los parámetros iterativamente usando el método de descenso de gradiente para minimizar la función de pérdida. Esto es posible ya que todas las operaciones que se realizan en la red son derivables.

El descenso de gradiente consiste en calcular la primera derivada (gradiente) de la función de pérdida, para así conocer la pendiente en ese punto. En cada iteración, una vez que todas las neuronas disponen de su valor de gradiente correspondiente, se actualizan los valores de los parámetros en el sentido contrario al que indica el gradiente, reduciendo así la función de pérdida.

La relación entre los valores del parámetro y el error, y el efecto del método del descenso de gradiente se puede representar con el siguiente esquema:

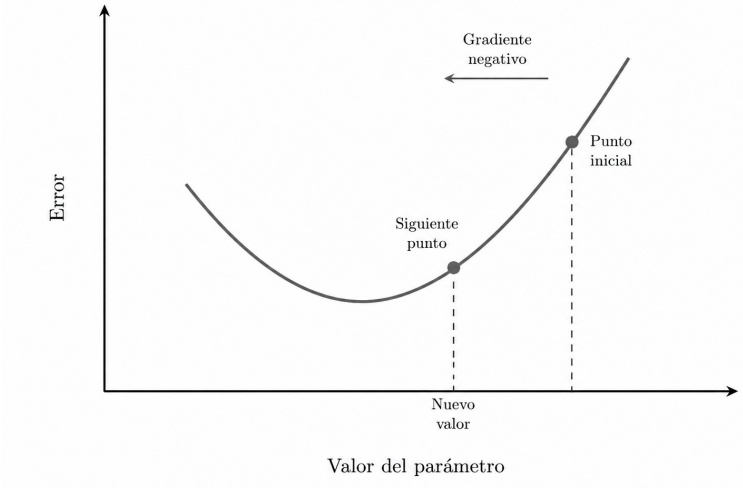


Figura 27: Esquema del descenso de gradiente.

La tendencia que se aprecia en la Figura 27 no es la de este proyecto, pero se usa para el buen entendimiento de lo que se está explicando.

El optimizador `Adam`, usa el descenso de gradiente estocástico, en el que los datos (del Dataset) son seleccionados y pasados por la red de manera aleatoria. Esta aleatoriedad, implica muchas veces una mejor generalización, como ocurre en este proyecto.

Luego, los algoritmos de descenso de gradiente multiplican la magnitud del gradiente por un escalar, conocido como rango de aprendizaje (`learning_rate`), y es el valor del hiperparámetro mostrado en el código. Básicamente, controla a qué distancia se quiere el siguiente valor del parámetro tras el ajuste. En este caso, ha sido apropiado un valor de 0.0001, ya que se requería de gran precisión.

El último argumento de `.compile()` es la métrica (`metrics`) y el hiperparámetro usado es, de nuevo, `mae`. Esto es usado para monitorizar el proceso de aprendizaje y prueba, y poder evaluar si las decisiones tomadas (hiperparámetros elegidos como el número de neuronas o aquellos indicados dentro del método `.compile()`) han sido las correctas o hay que hacer algún cambio. Se verá más adelante.

4.5. Entrenamiento del modelo.

Una vez definido el modelo y configurado su método de aprendizaje ya puede darse lugar a la fase de entrenamiento del modelo.

Para ello, se invoca el método `.fit()` como se muestra a continuación:

```
3         history = model.fit(
4             input_train_norm,
5             output_train_norm,
6             validation_split=0.2,
7             verbose=1,
8             epochs=1000,
9             callbacks=[early_stop, lr_scheduler]
10            )
```

En el primer argumento, se almacenan los datos de entrada que previamente se habían separado para el entrenamiento (`input_train_norm`), y en el segundo las salidas (`output_train_norm`), en nuestro problema, las dinámicas de las variables del reactor.

El tercer argumento es el llamado `validation_split`, el cual, de los datos de entrenamiento, separa un 20 % de esos datos para la validación del modelo. Un esquema visual de la separación de datos en este punto del proceso sería el siguiente:

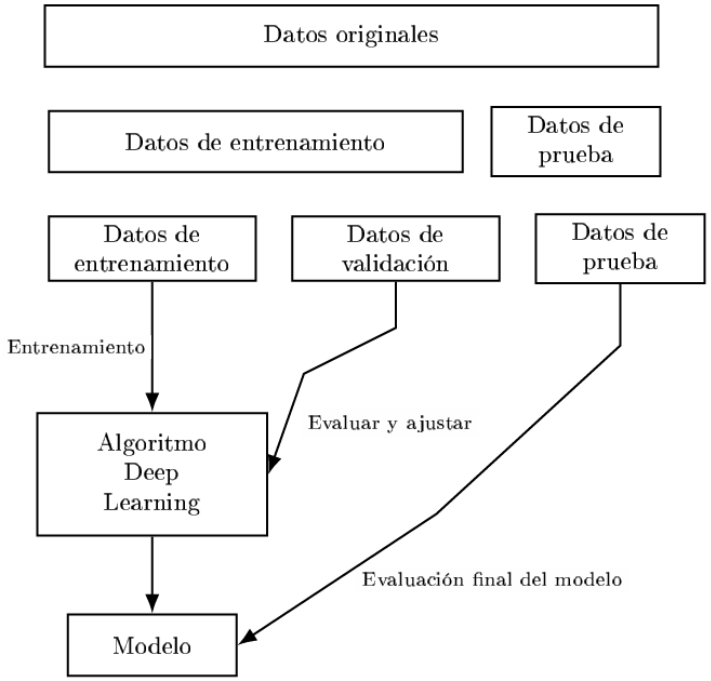


Figura 28: Esquema de la separación de datos realizada.

La validación del modelo consiste en comparar los errores por época entre los datos con los que ha sido entrenado el modelo y datos que no ha visto todavía (de validación), para saber cómo de bien generaliza el modelo. De esta forma, se van cambiando los hiperparámetros hasta acertar con unos que proporcionen buena generalización.

Lo esperable es que el MAE de los datos de entrenamiento y validación sigan una tendencia parecida, como se aprecia en el siguiente gráfico:

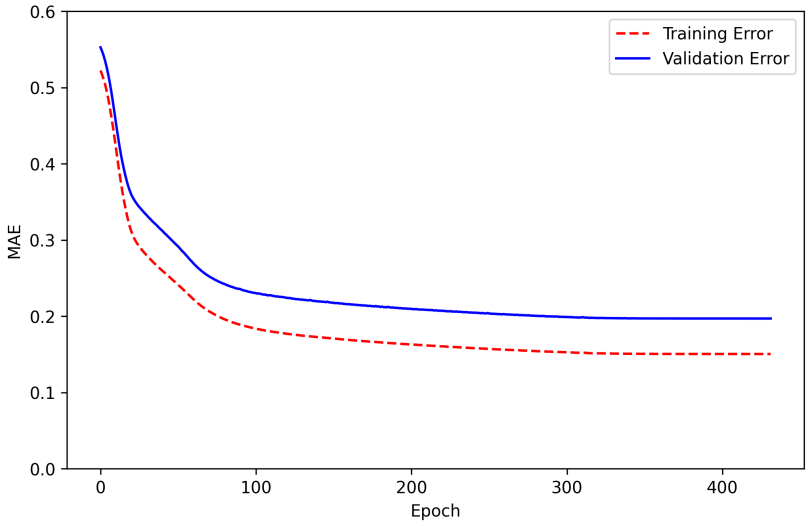


Figura 29: Progersión del error por época entre datos de entrenamiento y validación.

Se aprecia como los datos de entranamiento han tenido un error menor. Esto es coherente

ya que son los datos con los que se ha entrenado.

Además, tras cada época, el error disminuye, esto quiere decir que la red está aprendiendo y reprogramando los parámetros de manera que vaya mejorando la capacidad predictiva. Entiéndase por época (epoch) el número de veces que los datos pasan por la red.

El cuarto argumento es `verbose` y dándole el valor de 1 se obliga a que muestre cómo va avanzando el proceso cuando ejecutamos el código.

Luego, se indica el número de épocas totales que deberían llevarse a cabo con el argumento `epochs`, dándole a su hiperparámetro un valor de 1000. No llega a esa época, sino que se han necesitado 432 épocas para que el error se mantenga estable y no haya mejoría:

```
Epoch 430/1000
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1599 - mae: 0.1505 - val_loss: 0.2064 - val_mae: 0.1970 - learning_rate: 1.0000e-06
Epoch 431/1000
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1599 - mae: 0.1505 - val_loss: 0.2064 - val_mae: 0.1970 - learning_rate: 1.0000e-06
Epoch 432/1000
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1599 - mae: 0.1505 - val_loss: 0.2064 - val_mae: 0.1970 - learning_rate: 1.0000e-06
```

Figura 30: Últimas 3 épocas.

La Figura 30 es una captura de pantalla del resultado del código ejecutado en el que se muestran las 3 últimas épocas y sus errores (esto se muestra gracias a `verbose`).

El motivo por el cual no alcanza las 1000 épocas, es debido al último argumento llamado `callbacks`, que añade restricciones.

La primera de ellas es `EarlyStopping`:

```
16        early_stop = tf.keras.callbacks.EarlyStopping(
17                            monitor='val_loss',
18                            patience=5,
19                            restore_best_weights=True
20                            )
```

Este código hace un seguimiento de la métrica `val_loss`. Si durante 5 épocas (indicado en `patience`) no disminuye su valor, el modelo no mejora y debe parar el proceso de entrenamiento.

La segunda restricción es `lr_shceduler`, que usa el método `.ReduceLROnPlateau`. Con ello, si tras 5 épocas no ha mejorado, en lugar de parar el proceso, disminuye el ratio de aprendizaje (learning rate) progresivamente, hasta un mínimo de 10^-6

```
16        lr_scheduler = tf.keras.callbacks.ReduceLROnPlateau(
17                        monitor='val_loss',
18                        factor=0.5,
19                        patience=5,
20                        min_lr=1e-6,
21                        verbose=1
```

```
22                                  )
```

Nótese que ambas restricciones se complementan. Hasta que no se haya reducido el ratio de aprendizaje hasta la cifra mencionada no se activa `early_stop`.

Los hiperparaémtros escogidos, han sido seleccionados tras múltiples ejecuciones del código. Teóricamente, añadir más capas aumenta la capacidad del modelo, y esto es verdad, pero, en este caso, por mucho que se aumentaran el número de capas, los resultados obtenidos eran muy similares.

En ocasiones, para mejorar las predicciones, no solo es necesario afinar los hiperparámtros o añadir nuevos (existen muchos más de lo que se han utilizado en este proyecto), sino que las limitaciones pueden tener origen en el Dataset. Más adelante se dedicará parte de este texto a las limitaciones del proyecto.

4.6. Evaluación con los datos de prueba.

Antes de hacer las predicciones, queda comprobar lo bien que generaliza el modelo al usar el conjunto de datos de prueba (`input_test_norm` y `output_test_norm`). Para ello, se usa el método `.evaluate()`. En definitiva, se usa para medir el rendimiento final del modelo después de ser entrenado midiendo las métricas de error:

```
1         loss, mae = model.evaluate(
2                     input_test_norm,
3                     output_test_norm,
4                     verbose=2
5                     )
6
7                     print("Test Loss:", loss)
8                     print("Test MAE:", mae)
```

Con este código, la red toma los datos de entrada, hace predicciones, compara las predicciones con sus valores reales y calcula las métricas de error.

La variable `loss` y `mae` realmente son lo mismo, pero internamente se guardan en estas variables valores distintos ya que se utiliza el regularizador L2 y esto hace que `loss = MAE + penalización L2` ý `mae = MAE`. Es por ello que no se obtienen los mismo resultados:

```
Test Loss (MSE): 0.17673055827617645
Test MAE: 0.16762322187423706
```

En cuanto al error obtenido, se puede considerar apropiado y que generaliza bien. En el siguiente apartado se profundiza más en las predicciones realizadas.

5. RESULTADOS.

Finalmente, y sin más dilación, se hacen las predicciones. Para ello, se usa el método `.predict()`, y como datos de entrada se usa `input_test_norm`, que se han utilizado para la evaluación pero que el modelo no ha llegado a ver:

```
1        y_pred = model.predict(input_test_norm)
```

En este paso, los datos de entrada pasan por la red neuronal, llevándose a cabo las operaciones requeridas con los pesos y sesgos de cada neurona que se han ajustado durante el entrenamiento, siendo estos constantes, hasta llegar a la salida en la que se obtienen las dinámicas predichas por el modelo.

A continuación, se muestran algunos ejemplos de predicciones exitosas y predicciones fallidas.

Predicciones exitosas.

Simulación 109:

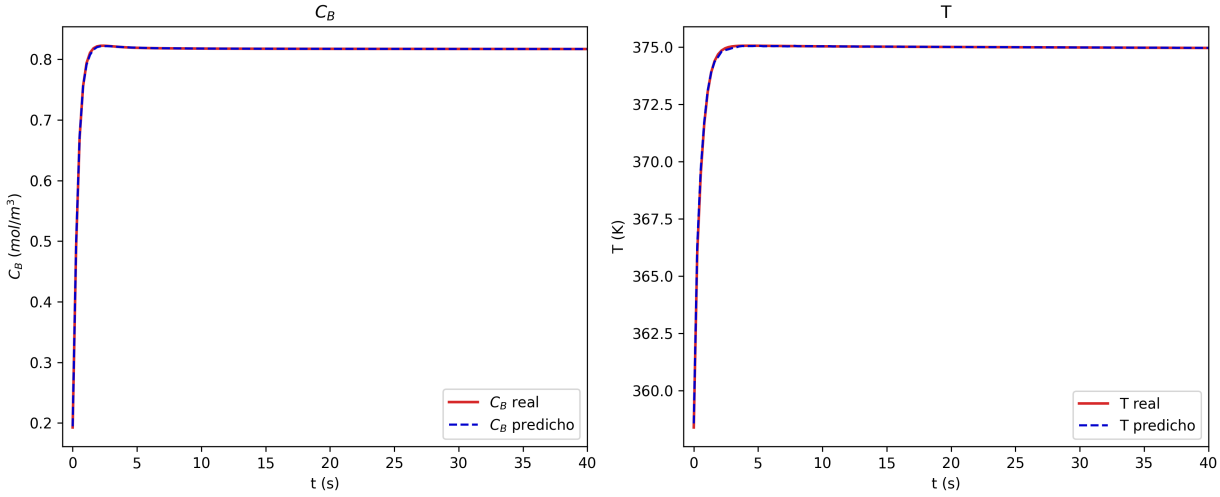


Figura 31: Dinámicas de C_B y T. Simulación 109.

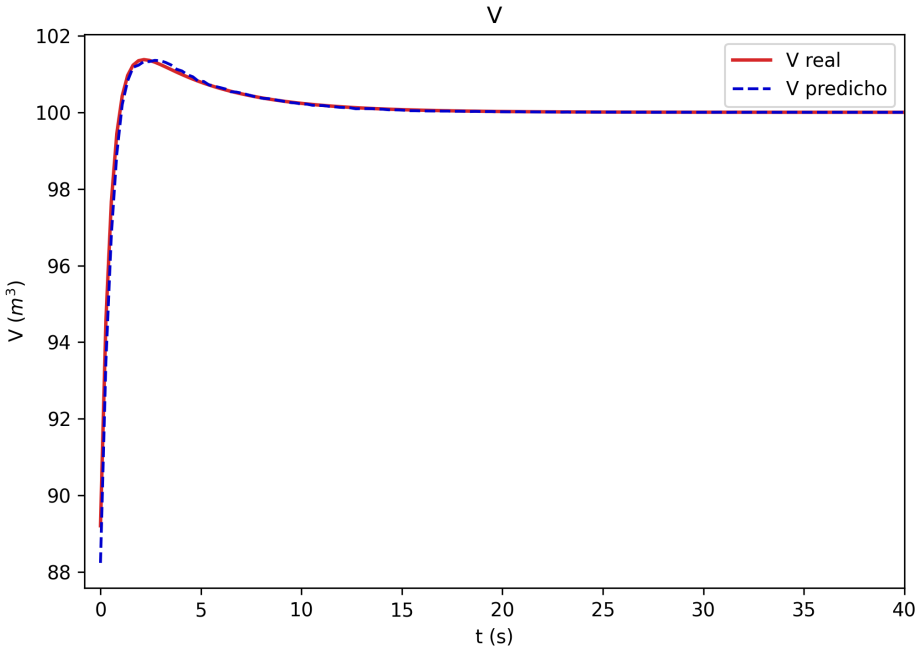


Figura 32: Dinámica de V. Simulación 109.

Simulación 75:

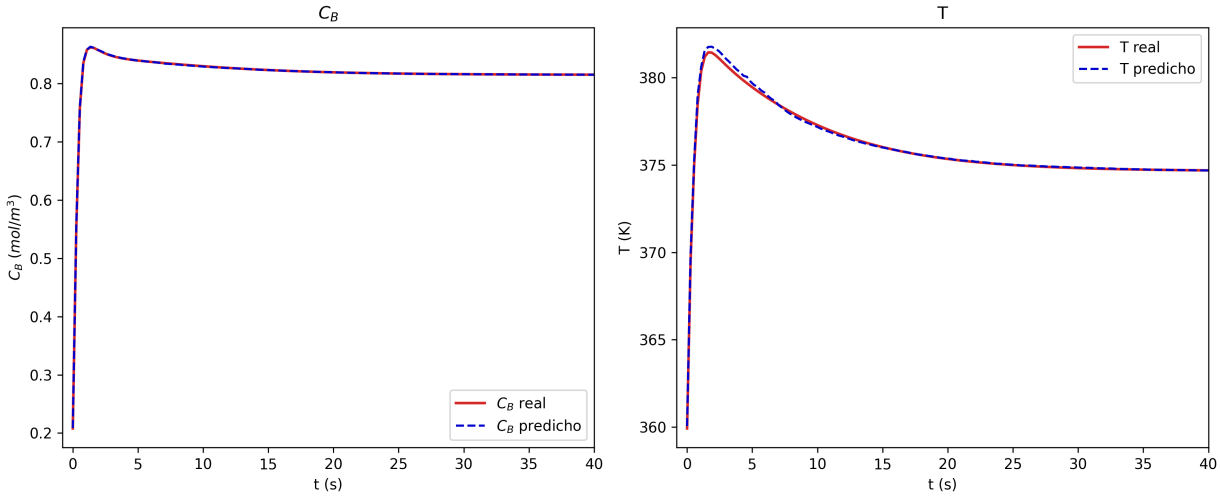


Figura 33: Dinámicas de C_B y T. Simulación 75.

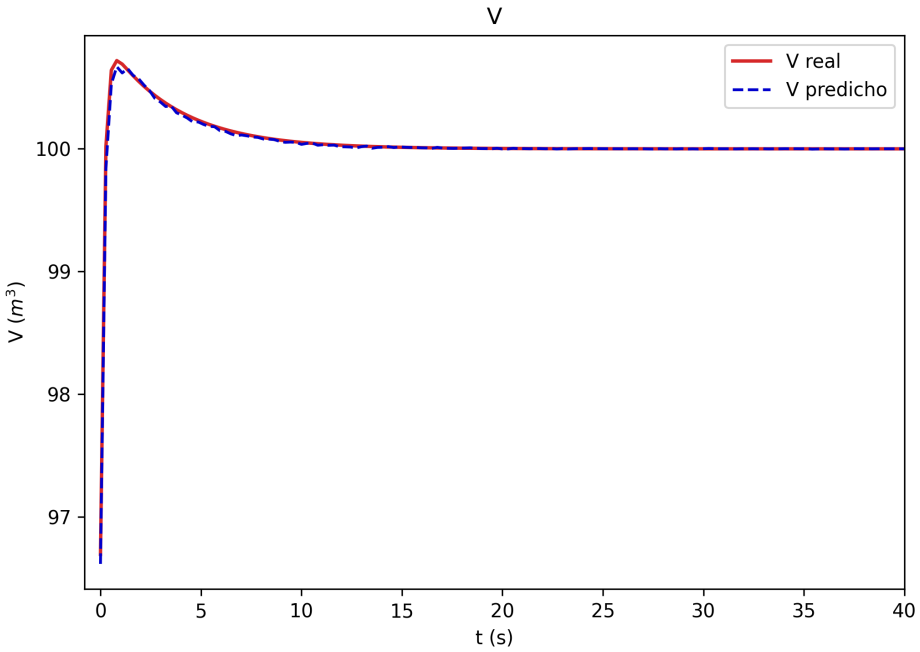


Figura 34: Dinámica de V. Simulación 75.

Simulación 57:

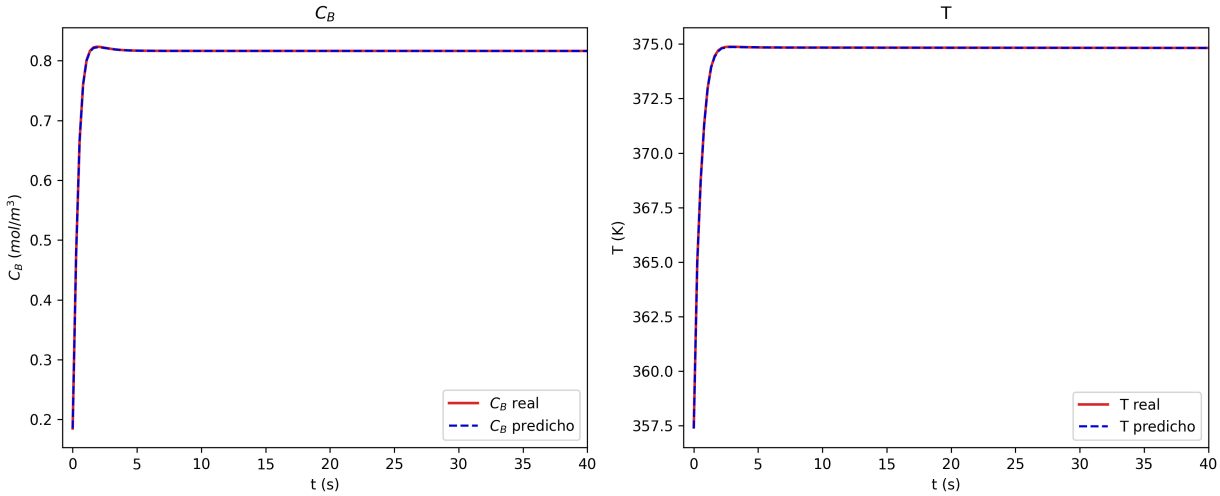


Figura 35: Dinámicas de C_B y T. Simulación 57.

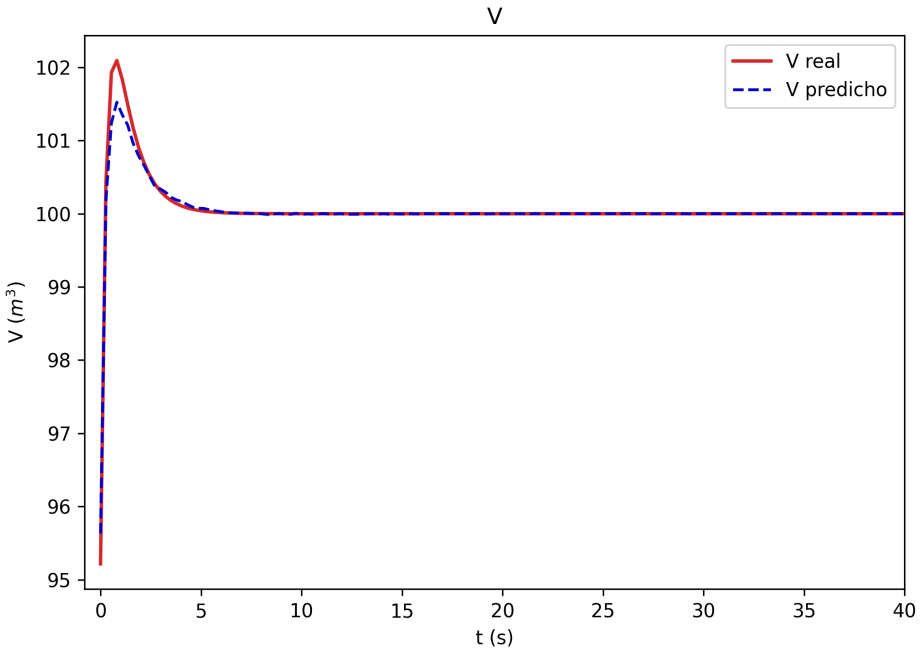


Figura 36: Dinámica de V. Simulación 57.

Simulación 125:

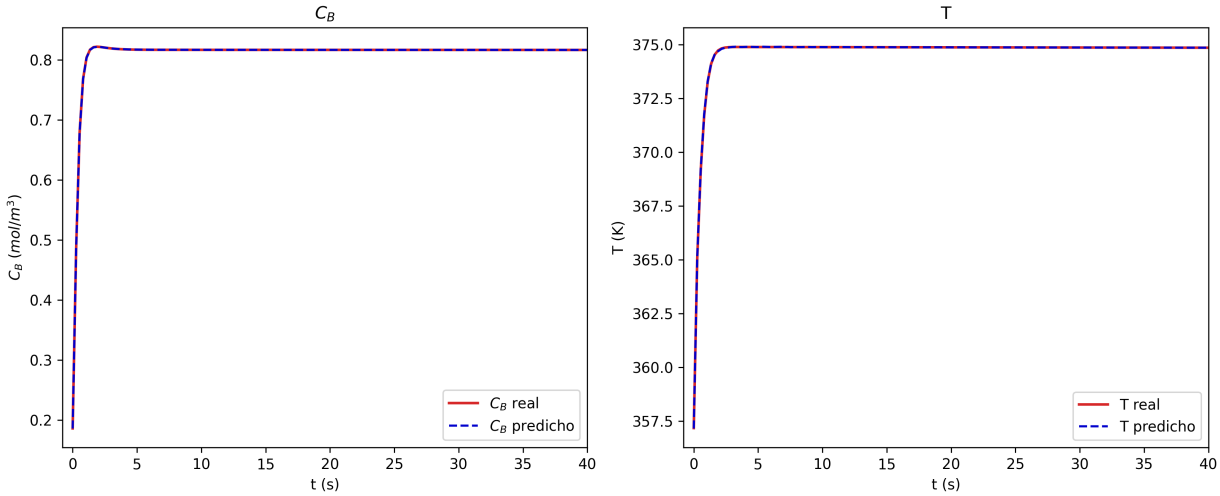


Figura 37: Dinámicas de C_B y T. Simulación 125.

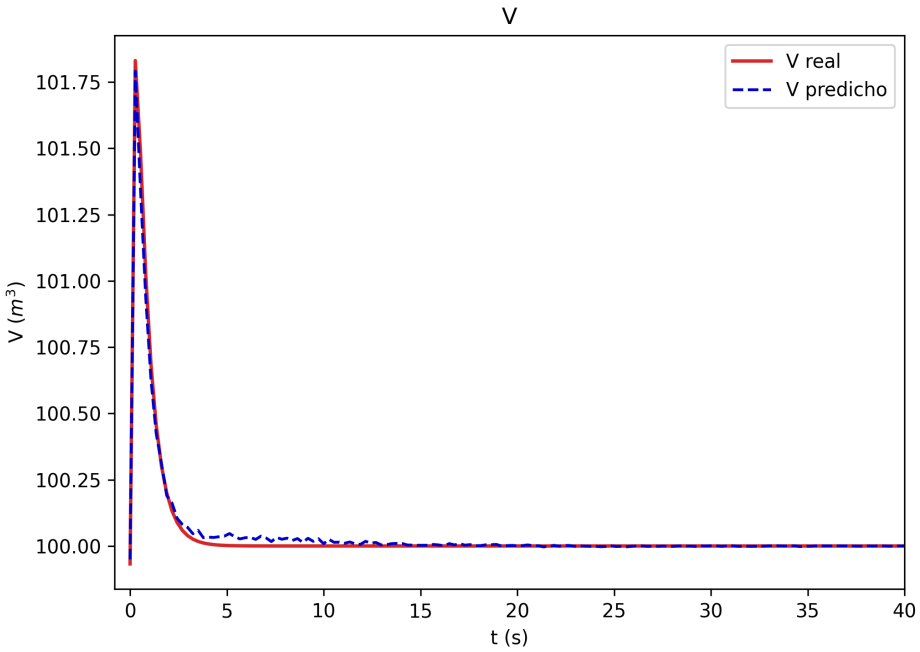


Figura 38: Dinámica de V. Simulación 125.

Simulación 218:

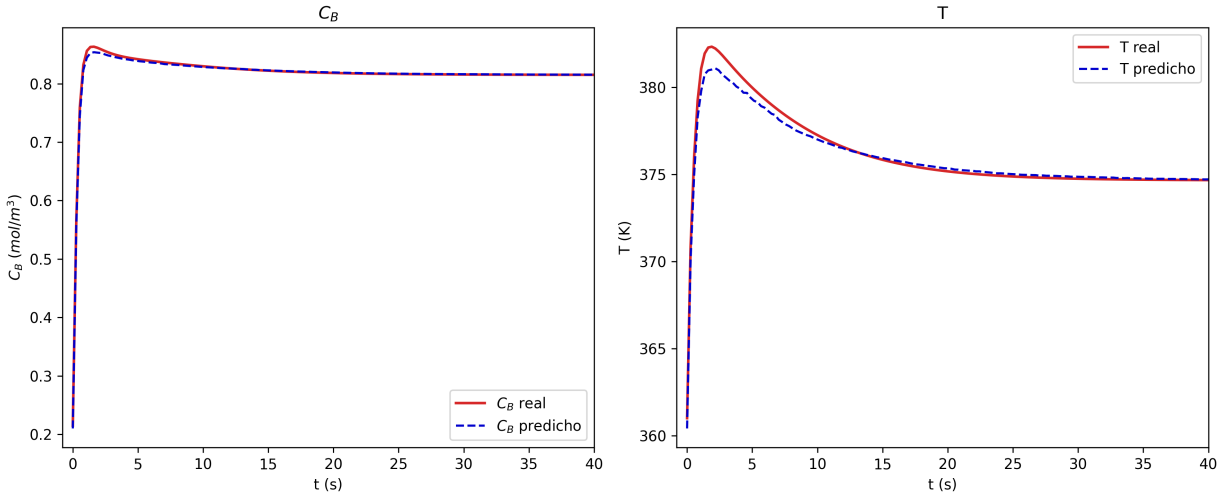


Figura 39: Dinámicas de C_B y T. Simulación 218.

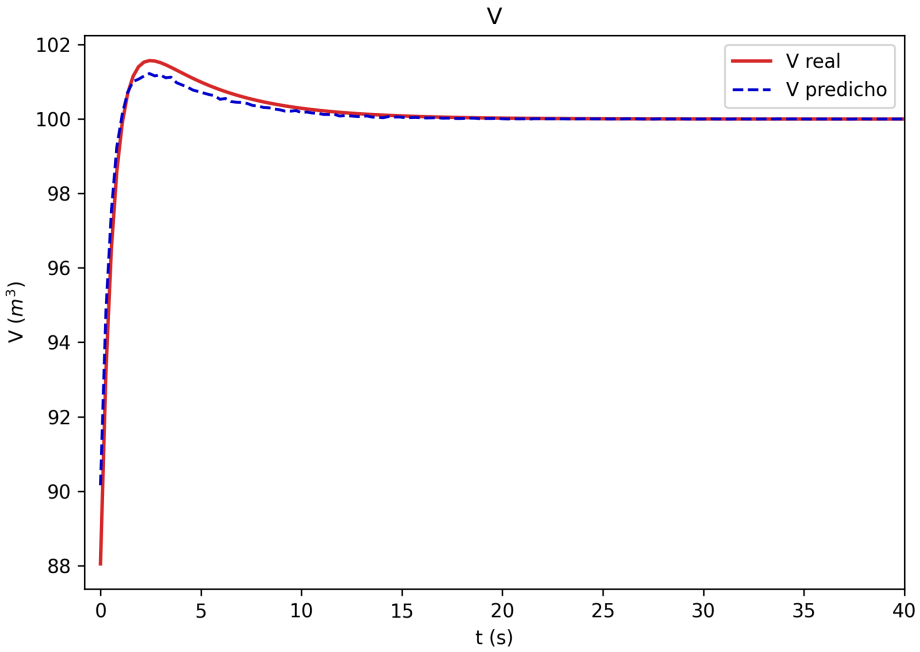


Figura 40: Dinámica de V. Simulación 218.

Predicciones fallidas.

Simulación 243:

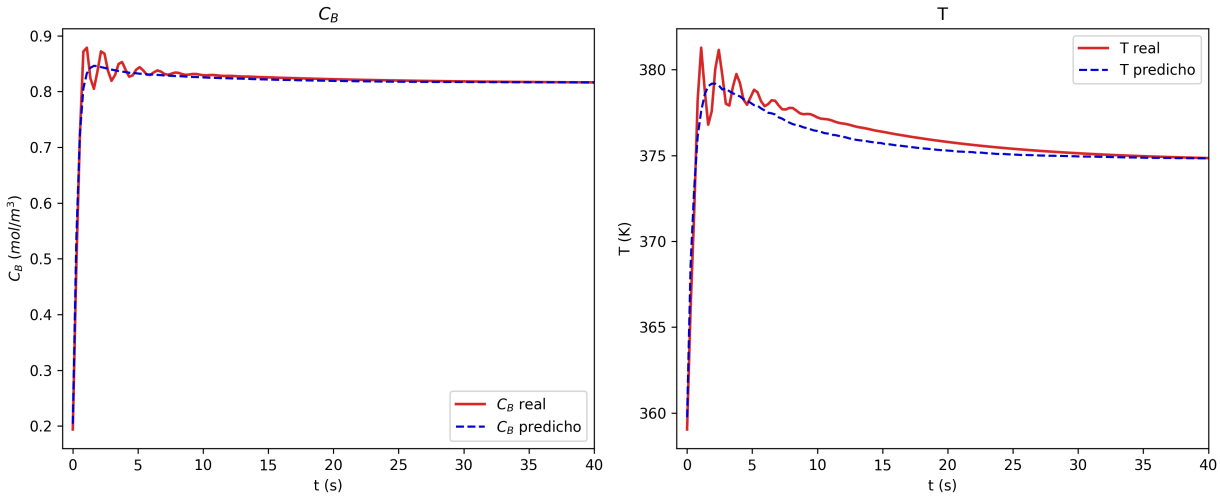


Figura 41: Dinámicas de C_B y T. Simulación 243.

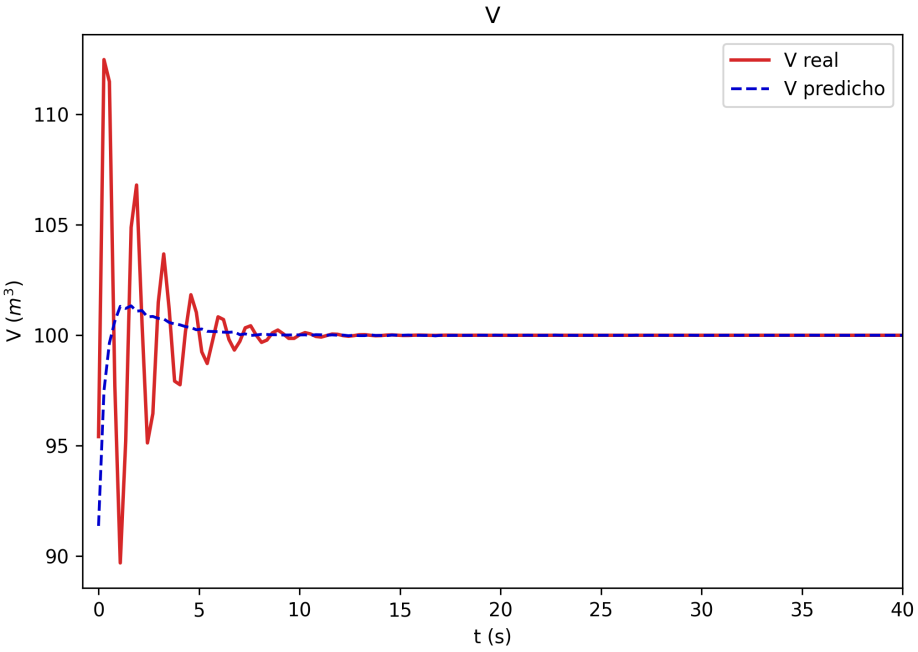


Figura 42: Dinámica de V. Simulación 243.

Simulación 208:

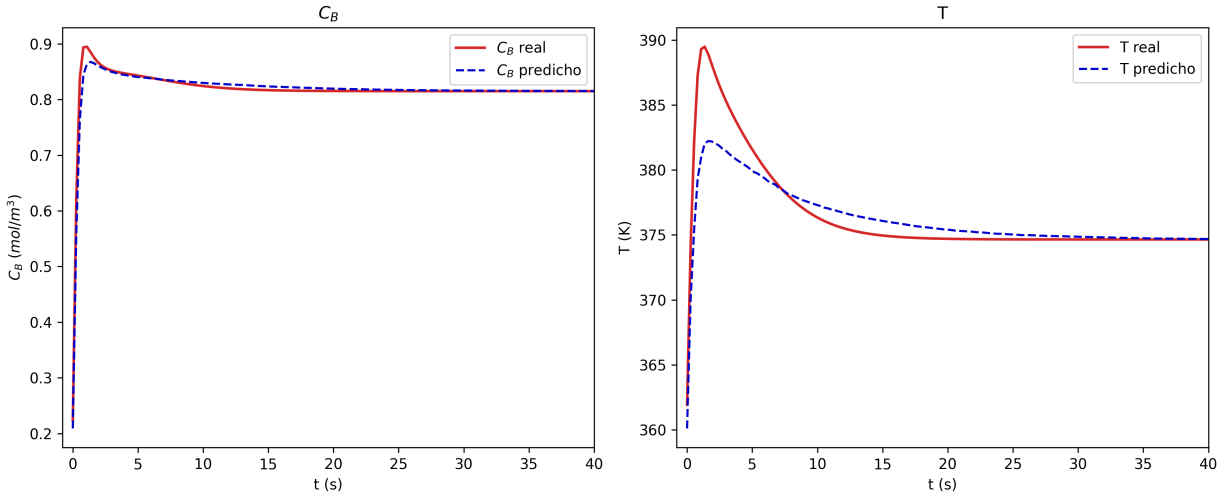


Figura 43: Dinámicas de C_B y T. Simulación 208.

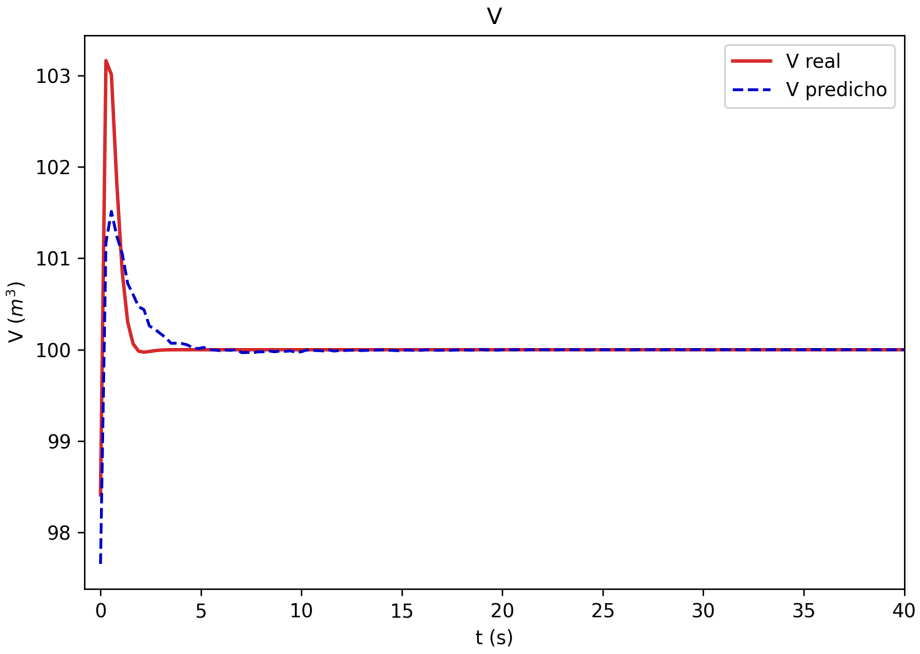


Figura 44: Dinámica de V. Simulación 208.

Simulación 251:

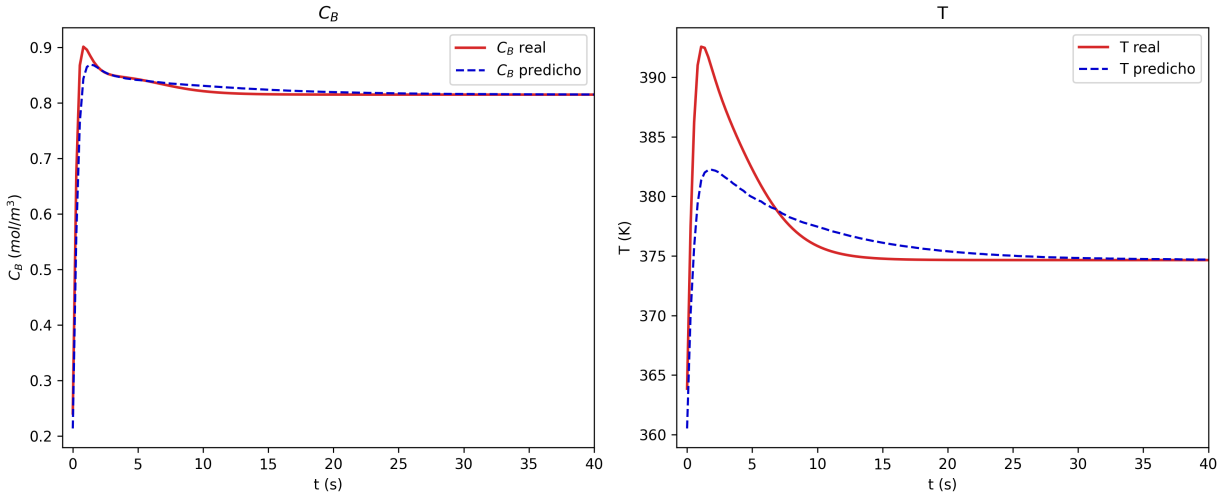


Figura 45: Dinámicas de C_B y T. Simulación 208.

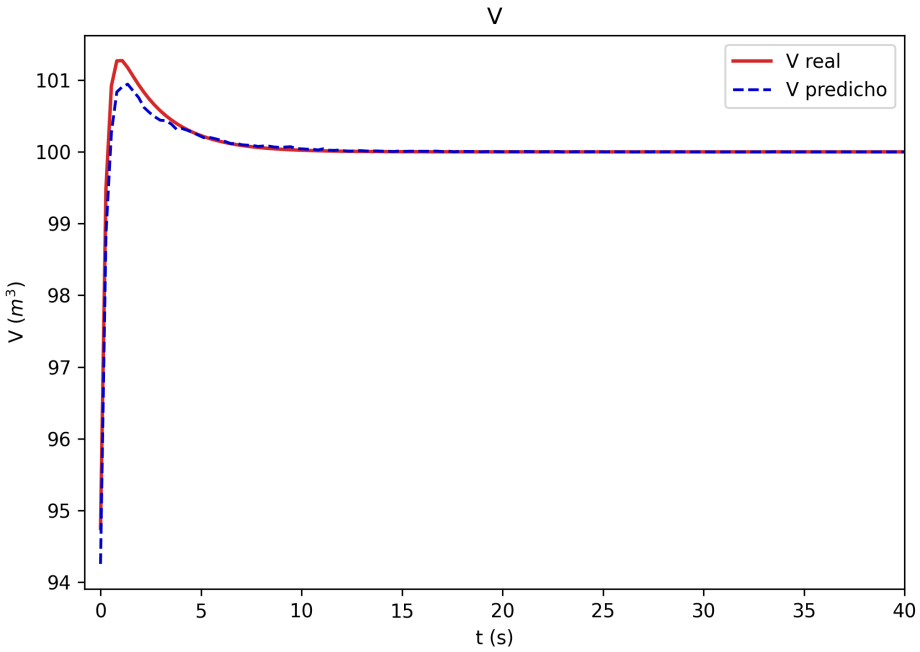


Figura 46: Dinámica de V. Simulación 251.

La mayoría de predicciones, han seguido una tendencia similar a las dinámicas reales. No obstante, algunas de ellas que implican una gran complejidad debido a la alta oscilación, la red ha probado ser incapaz de predecirlas. Esta limitación y otras, se comentan en el siguiente apartado.

6. LIMITACIONES Y FUTURAS MEJORAS.

La red neuronal, a pesar de haber dado unos resultados adecuados, tiene ciertas limitaciones tanto por la capacidad de predicción que ofrece el modelo, como las decisiones tomadas durante todo el proyecto hasta la cosntrucción de dicho modelo.

En primer lugar, todo se ha desarrollado en base a un solo problema, es decir, un RCTA en el que los estados estacionarios siempre son los mismos. La red, sí que ha aprendido las dinámicas de las variables en los primeros segundos de las simulaciones, al haber muchas dinámicas distintas, pero, al final, todas ellas confluyen en los mismos estados estacionarios, por tanto, la red no ha aprendido, sino que ha memorizado que cada variabe, debe llegar al valor calculado de su estado estacionario (3.2). Además, no solo no se ha dado la capacidad a la red de aprender otros setpoints, sino que no se han introducido distintos valores iniciales sobre los que las simulaciones empiecen a llevarse a cabo, ni perturbaciones a lo largo de ellas, de ahí, que sea todo un mismo problema.

Sin embargo, esto no es negativo. La capacidad del modelo y aquello que tiene que aprender, puede ser escogido y elaborado por el propio usuario. Es posible crear otros Datasets en los que, además de cambiar los parámtros de los PIs, se pueden crear nuevos escenarios, ofreciendo variabiliad en los estados iniciales e incluso añadiendo perturbaciones. Para ello, se habría necesitado un trabajo de elaboración de datos muchos más exhastivo, e incluso la creación e incorporación de nuevos Datasets en los que cada uno de ellos ofrezca unas características distintas para que el modelo las aprenda.

Con este proyecto, se demuestra la posibilidad de poder hacerlo, además de impulsar futuros proyectos mucho más complejos y rigurosos. Todo ello, implicaría también una nueva definición del modelo y la implementación de nuevas técnicas para su correcto desempeño.

Luego, la variedad de simulaciones, se ha ofrecido por medio del cambio de los parámetros del controlador dentro de unos rangos acortados. Esto quiere decir, que la red no aprende comportamientos del reactor en los que los parámetros de los PIs se encuentren fuera de esos rangos (Tabla 4, 4.1).

Los rangos, se eligieron por medio de un proceso de prueba y error en base a la información proporcionada por el artículo que se ha usado de base, en el que, fuera de ellos, la cantidad de simulaciones que no llegaban al estado estacionario incrementaba significativamente. De ahí, deriva otra limitación importante: La red no aprende errores.

Si se quisiera ser lo más fiel posibile a un reactor real, la red debería aprender con qué combinaciónes de parámetros de los PIs no se alcanzan estados estacionarios, (lo que se ha definido en este proyecto como simulaciones fallidas), pues dichos errores, forman

parte también de un comportamiento real del reactor.

La solución, de nuevo, sería recoger y proporcionar dichos errores en forma de datos numéricos para que los aprende como nuevas características, por ejemplo, en el preceso de filtrado (4.3). Dependerá del objetivo final al que desee llegar el usuario.

Por otro lado, véase también como el tiempo no esta incluido de manera explícita en los datos (Tabla 6, 4.4), sino que cada columna, es 1 punto de los 150 (Apéndice D.1, línea 60) en los que se han repartido las simulaciones. De hecho, se ha hecho el cambio de estructura del Dataset mencionada con esa finalidad, pues facilitaba el proceso aprendizaje aportando además soluciones coherentes como se ha visto en las predicciones.

Esto es debido a que para un valor de tiempo, se puede ofrecer cualquier dato, al ser la variable independiente. Para poder añadir el tiempo dentro del aprendizaje se podría haber usado la estructura original (Tabla 5, 4.4), pero, de nuevo, se habrían necesitado muchísimos más datos.

Otra limitación mencionada en el apartado anterior, es que la red ha sido incapaz de predecir dinámicas complejas, como aquellas que tienen gran oscilación. Otro Dataset en el que se recogieran todas aquellas que presentan esa oscilación habría podido mejorar esas predicciones. Otras no son tan complejan pero no son comunes dentro del rango aplicado, reduciendo también la capacidad de predecirlas.

En cuanto a la propia red nuronal, los hiperparámetros elegidos han sido apropiados cumplir con los objetivos de este proyecto. Aún así, éstos pueden variar, tanto para este proyecto como para proyectos con las mejorías propuestas. A continuación, se aporta una lista de hiperparámtros que pueden usarse y podrían ser también útiles:

Tabla 9: Hiperparámetros utilizados en la red neuronal y posibles alternativas viables.

Hiperparámetro	**Valor utilizado**	**Otros valores/opciones viables**
Número de neuronas capa 1	128	32, 64, 256, 512
Función de activación	relu	tanh, sigmoid, elu, selu, leaky_relu
Regularización L2	0.0001	0, 10^{-5}, 10^{-3}, 10^{-2}, L1

Hiperparámetro	**Valor utilizado**	**Opciones viables**
Número de neuronas capa 2	64	32, 128, 256
Número de capas ocultas	2	1, 3, 4
Optimizador	Adam	SGD, RMSprop, AdamW, Nadam
Learning rate inicial	0.0001	10^{-2}, 10^{-3}, 5×10^{-4}, 10^{-5}
Función de pérdida (loss)	mae	mse, huber, logcosh
Métrica (metrics)	mae	mse, rmse, mape
Número máximo de épocas (epochs)	1000	100, 300, 500, 2000
Validation split	0.2	0.1, 0.15, 0.25, 0.3
Variable monitorizada	val_loss	val_mae, loss
Patience de EarlyStopping	5	10, 20, 50
Restore best weights	True	False
Factor de reducción del learning rate	0.5	0.1, 0.2, 0.8
Learning rate mínimo (min_lr)	10^{-6}	10^{-5}, 10^{-7}, 10^{-8}
Verbose entrenamiento	1	0, 2
Verbose evaluación	2	0, 1

En ls siguiente tabla, se muestran aquellos hiperparámetros que no se han usado y podrían ser también útiles.

Tabla 10: Otros hiperparámetros potencialmente útiles para la red neuronal.

Hiperparámetro	**Ejemplo**	**Breve explicación**
Batch size	32	Define el número de muestras utilizadas antes de actualizar los pesos de la red neuronal.
Dropout	0.2	Desactiva neuronas aleatoriamente durante el entrenamiento para reducir el sobreajuste.
Kernel initializer	he_normal	Método empleado para inicializar los pesos iniciales de la red neuronal.
Momentum	0.9	Permite acelerar el descenso de gradiente acumulando parte de actualizaciones anteriores.
Beta_1 (Adam)	0.9	Controla el suavizado exponencial del promedio de gradientes en Adam.
Beta_2 (Adam)	0.999	Controla el suavizado exponencial del promedio de gradientes cuadrados en Adam.
Shuffle	True	Determina si los datos se mezclan aleatoriamente en cada época de entrenamiento.
Gradient clipping	1.0	Limita el valor máximo de los gradientes para evitar inestabilidades numéricas.
Activación de salida	lineal	Define la función de activación utilizada en la capa de salida de la red neuronal.

7. CONCLUSIONES.

Se da fin a este proyecto con este último apartado, en el que se extraen las conclusiones.

Primeramente, se ha demostrado que una red neuronal, conlleva un trabajo extenso previo de creación (o búsqueda en otros casos) de datos que describan con verosimilitud el caso de estudio concreto.

Para este caso, ha ido desde la elección de los estados estacionarios deseados, hasta la elección de un Dataset con una esctructura específica, pasando por un muestreo para la obtención de distintas simulaciones posibles cambiando los parámetros, numerosas veces mencionados, de los PIs.

Tras este trabajo la red neuronal ha hecho, dentro de sus limitaciones, buenas predicciones, obteniendo dinámicas muy parecidas e incluso iguales a las dinámicas reales, siendo imposible predecir aquellas que presentaban muchas variaciones en el transcurso del tiempo.

Por ende, el trabajo de creación de datos, también ha sido fructífero y puede constituir un método de trabajo sobre el que apoyarse para futuros trabajos más rigurosos.

Por ello, además, se cuestiona el potencial que pueden llegar a tener las redes neuronales utilizando Tensorflow + Keras, si se hiciera un trabajo más selectivo y organizado de los datos, para que aprenda otras características y pueda predecirlas, como las dinámicas con mucha oscilación. O incluso, utilizando otras técnicas o hiperparámetros de Deep Learning.

Referencias

[1] Bloor, M., Ahmed, A., Kotecha, N., Mercangöz, M., Tsay, C., & del Río-Chanona, E. A. (2025). *Control-informed reinforcement learning for chemical processes*. Industrial & Engineering Chemistry Research, 64, 4966–4978. https://doi.org/10.1021/acs.iecr.4c03233

[2] SciPy Developers. (2025). *scipy.stats.qmc.Sobol*. SciPy Documentation. https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.qmc.Sobol.html

[3] TensorFlow Developers. (2025). *Adam optimizer*. TensorFlow Documentation. https://www.tensorflow.org/api_docs/python/tf/keras/optimizers/Adam

[4] Torres, J. (2021). *Python Deep Learning: Introducción práctica con Keras y TensorFlow* (2.ª ed.). Marcombo.

[5] Fons, I. (2026). *Introducción al Machine Learning* [Presentación]. Grupo COnCEPT, Departamento de Ingeniería Química, Universidad de Alicante.

APÉNDICE

A. CÁLCULO ESTADOS ESTACIONARIOS

```
import numpy as np
import matplotlib.pyplot as plt
import math
from scipy.optimize import fsolve

# ==========================
# Parametros del proceso
# ==========================
Tf    = 350                # K
Cain  = 1                  # mol/m3
Fout  = 100                # m3/s
rho   = 1000               # kg/m3
Cp    = 0.239              # J/(kg*K)
UA    = 5e4                # W/K
Ha    = 5e3                # J/mol
EaR   = 8750               # K
ka    = 7.2e10             # s^-1
Hb    = 4e3                # J/mol
EbR   = 10750              # K
kb    = 8.2e10             # s^-1

# ==========================
# Condiciones fijas
# ==========================
V_s = 100.0
Fin_s = Fout

# ==========================
# Barrido en Tc_s
# ==========================
Tc_vals = []
T_vals  = []
Ca_vals = []
Cb_vals = []
Cc_vals = []

# Guess inicial: [Ca_s, Cb_s, Cc_s, T_s]
guess = [0.8, 0.1, 0.05, 330.0]

for Tc_s in range(290, 451):

def steady_state(vars_):
```

```
        Ca_s, Cb_s, Cc_s, T_s = vars_

        Teff = max(T_s, 1.0)

        ra = ka * math.exp(-EaR / Teff) * Ca_s
        rb = kb * math.exp(-EbR / Teff) * Cb_s

        f = np.zeros(4)

        # dCb/dt = 0
        f[0] = ra - rb - Fin_s * Cb_s / V_s

        # dT/dt = 0
        f[1] = (
        Fin_s * (Tf - T_s) / V_s
        - Ha * ra / (rho * Cp)
        - Hb * rb / (rho * Cp)
        + UA * (Tc_s - T_s) / (V_s * rho * Cp)
        )

        # dCa/dt = 0
        f[2] = Fin_s * (Cain - Ca_s) / V_s - ra

        # dCc/dt = 0
        f[3] = rb - Fin_s * Cc_s / V_s

        return f

        sol = fsolve(steady_state, guess)
        Ca_s, Cb_s, Cc_s, T_s = sol

        # Actualizar guess para mejorar convergencia
        guess = sol

        Tc_vals.append(Tc_s)
        T_vals.append(T_s)
        Ca_vals.append(Ca_s)
        Cb_vals.append(Cb_s)
        Cc_vals.append(Cc_s)

        # ==========================
        # Convertir a arrays
        # ==========================
        Tc_vals = np.array(Tc_vals, dtype=float)
        T_vals  = np.array(T_vals, dtype=float)
        Ca_vals = np.array(Ca_vals, dtype=float)
        Cb_vals = np.array(Cb_vals, dtype=float)
```

```
90          Cc_vals = np.array(Cc_vals, dtype=float)
91
92          # ========================
93          # Maximo de Cb_s
94          # ========================
95          idx_max = np.argmax(Cb_vals)
96          Cb_max = Cb_vals[idx_max]
97
98          Tc_max = Tc_vals[idx_max]
99          T_max  = T_vals[idx_max]
100         Ca_max = Ca_vals[idx_max]
101         Cb_max_val = Cb_vals[idx_max]
102         Cc_max = Cc_vals[idx_max]
103
104         print("\n========================")
105         print("MAXIMO DE Cb_s")
106         print("========================")
107         print(f"Tc_max = {Tc_max:.2f} K")
108         print(f"T_max  = {T_max:.2f} K")
109         print(f"Ca_max = {Ca_max:.6f}")
110         print(f"Cb_max = {Cb_max_val:.6f}")
111         print(f"Cc_max = {Cc_max:.6f}")
112
113
114         # ========================
115         # SUBPLOTS
116         # ========================
117
118         import zipfile
119         from google.colab import files
120
121         # ========================
122         # FUNCION AUXILIAR
123         # ========================
124         def ajustar_solo_x(ax, x):
125         ax.set_xlim(np.min(x), np.max(x))
126         ax.margins(x=0)
127
128         # ========================
129         # FIGURA PRINCIPAL
130         # ========================
131         fig, axs = plt.subplots(2, 2, figsize=(12, 8))
132
133         # --- Subplot 1 ---
134         axs[0, 0].plot(Tc_vals, Cb_vals, label=r"$C_{B,s}$")
135         axs[0, 0].scatter(Tc_vals[idx_max], Cb_vals[idx_max],
136         label=r"Maximo $C_{B,s}$", s=60)
```

```
137
138         if idx_best is not None:
139         axs[0, 0].scatter(
140         Tc_vals[idx_best], Cb_vals[idx_best],
141         label="Compromiso prod-control", s=60
142         )
143
144         axs[0, 0].set_xlabel(r"$T_{c,s}$ (K)")
145         axs[0, 0].set_ylabel(r"$C_{B,s}$ (mol/m$^3$)")
146         axs[0, 0].set_title(r"$C_{B,s}$ vs $T_{c,s}$")
147         axs[0, 0].grid(False)
148         axs[0, 0].legend()
149         ajustar_solo_x(axs[0, 0], Tc_vals)
150
151         # --- Subplot 2 ---
152         axs[0, 1].plot(Tc_vals, dCb_dTc, label=r"$\frac{dC_B}{dT_c}$")
153         axs[0, 1].axhline(0, linestyle="--")
154
155         axs[0, 1].scatter(Tc_vals[idx_max], dCb_dTc[idx_max],
156         label=r"Maximo $C_{B,s}$", s=60)
157
158         if idx_best is not None:
159         axs[0, 1].scatter(
160         Tc_vals[idx_best], dCb_dTc[idx_best],
161         label="Compromiso prod-control", s=60
162         )
163
164         axs[0, 1].set_xlabel(r"$T_{c,s}$ (K)")
165         axs[0, 1].set_ylabel(r"$\frac{dC_B}{dT_c}$ (molm$^{-3}$K$^{-1}
               $)")
166         axs[0, 1].set_title("Sensibilidad")
167         axs[0, 1].grid(False)
168         axs[0, 1].legend()
169         ajustar_solo_x(axs[0, 1], Tc_vals)
170
171         # --- Subplot 3 ---
172         axs[1, 0].plot(Tc_vals, Ca_vals, label=r"$C_{A,s}$")
173         axs[1, 0].plot(Tc_vals, Cb_vals, label=r"$C_{B,s}$")
174         axs[1, 0].plot(Tc_vals, Cc_vals, label=r"$C_{C,s}$")
175
176         axs[1, 0].set_xlabel(r"$T_{c,s}$ (K)")
177         axs[1, 0].set_ylabel(r"Concentracion (mol/m$^3$)")
178         axs[1, 0].set_title(r"Concentraciones vs $T_{c,s}$")
179         axs[1, 0].grid(False)
180         axs[1, 0].legend()
181         ajustar_solo_x(axs[1, 0], Tc_vals)
182
```

```
        # --- Subplot 4 ---
        axs[1, 1].plot(Tc_vals, T_vals, label=r"$T_s$")

        if idx_best is not None:
        axs[1, 1].scatter(
        Tc_vals[idx_best], T_vals[idx_best],
        label="Compromiso prod-control", s=60
        )

        axs[1, 1].set_xlabel(r"$T_{c,s}$ (K)")
        axs[1, 1].set_ylabel(r"$T_s$ (K)")
        axs[1, 1].set_title(r"$T_s$ vs $T_{c,s}$")
        axs[1, 1].grid(False)
        axs[1, 1].legend()
        ajustar_solo_x(axs[1, 1], Tc_vals)

        plt.tight_layout()
        plt.show()
```

B. ESTUDIO DEL COMPORTAMIENTO DEL REACTOR.

B.1. Estudio general.

```
        Tc = 390
        Fin = 140

        # =========================
        # Parametros del proceso
        # =========================
        Tf    = 350                # K
        Cain  = 1                  # mol/m3
        Fout  = 100                # m3/s
        rho   = 1000               # kg/m3
        Cp    = 0.239              # J/(kg*K)
        UA    = 5e4                # W/K
        Ha    = 5e3                # J/mol
        EaR   = 8750               # K
        ka    = 7.2e10             # s^-1
        Hb    = 4e3                # J/mol
        EbR   = 10750              # K
        kb    = 8.2e10             # s^-1


```

```
        def CSTR_bloor(t, y):
        Cb, T, V, Ca, Cc = y

        ra = ka * math.exp(-EaR / T) * Ca
        rb = kb * math.exp(-EbR / T) * Cb

        dCb = ra - rb - Fin * Cb / V
        dT = Fin * (Tf - T) / V - Ha * ra / (rho * Cp) - Hb * rb / (
            rho * Cp) + UA / (V * rho * Cp) * (Tc - T)
        dV = Fin - Fout
        dCa = (Fin * (Cain - Ca)) / V - ra
        dCc = rb - (Fin * Cc) / V

        return [dCb, dT, dV, dCa, dCc]

        # Condiciones iniciales
        Cb0 = 0
        T0 = 327
        V0 = 10
        Ca0 = Cain
        Cc0 = 0

        t0 = 0
        tf = 25
        t_span = (t0, tf)

        y0 = [Cb0, T0, V0, Ca0, Cc0]

        sol = solve_ivp(CSTR_bloor, t_span, y0, method='RK45',
            dense_output=True)

        print(sol.success)
        print(sol.message)

        t = sol.t
        Cb = sol.y[0]
        T = sol.y[1]
        V = sol.y[2]
        Ca = sol.y[3]
        Cc = sol.y[4]
```

B.2. Estudio de la Temperatura.

```
        # =========================
        # Analisis termino a termino de dT/dt
        # =========================
```

```
4
5       import pandas as pd
6
7       # Recalcular velocidades de reaccion (vectorizadas)
8       ra = ka * np.exp(-EaR / T) * Ca
9       rb = kb * np.exp(-EbR / T) * Cb
10
11      # Terminos de la ecuacion de energia
12      term_flujo = Fin * (Tf - T) / V
13      term_ra = -Ha * ra / (rho * Cp)
14      term_rb = -Hb * rb / (rho * Cp)
15      term_UA    = UA * (Tc - T) / (V * rho * Cp)
16
17      # Suma total (dT/dt)
18      dTdt = term_flujo + term_ra + term_rb + term_UA
19
20      # ==========================
21      # DataFrame para inspeccion
22      # ==========================
23      df = pd.DataFrame({
24          "t": t,
25          "flujo": term_flujo,
26          "ra": term_ra,
27          "rb": term_rb,
28          "UA": term_UA,
29          "dTdt_total": dTdt
30      })
31
32      print(df.head())      # primeros valores
33      print(df.tail())      # ultimos valores
34
35
36
37      # ==========================
38      # FIGURA UNICA
39      # ==========================
40      plt.figure(figsize=(8,4))
41
42      # Terminos individuales
43      plt.plot(t, term_flujo, label="Flujo")
44      plt.plot(t, term_ra, label=r"$r_a$")
45      plt.plot(t, term_rb, label=r"$r_b$")
46      plt.plot(t, term_UA, label=r"$(T_c - T)$")
47
48      plt.xlabel("t (s)")
49      plt.ylabel("Contribucion a dT/dt (K/s)")
50      plt.title("Terminos de la ecuacion de energia")
```

```
51          plt.legend()
```

C. COMPORTAMIENTO DEL REACTOR CON CONTROLADOR PI

```
1           import numpy as np
2           from scipy.integrate import solve_ivp
3           import matplotlib.pyplot as plt
4           import math
5
6
7
8           # ==========================
9           # Parametros del proceso
10          # ==========================
11          Tf    = 350
12          Cain  = 1
13          Fout  = 100
14          rho   = 1000
15          Cp    = 0.239
16          UA    = 5e4
17          Ha    = 5e3
18          EaR   = 8750
19          ka    = 7.2e10
20          Hb    = 4e3
21          EbR   = 10750
22          kb    = 8.2e10
23
24          # ==========================
25          # Comprobacion basica
26          # ==========================
27          if Tc_opt is None:
28          raise ValueError("No existe punto de compromiso. Ejecuta antes
                el codigo de estados estacionarios.")
29
30          # ==========================
31          # Setpoints y bias desde el punto optimo
32          # ==========================
33          Cb_s  = Cb_opt
34          V_s   = 100.0
35          T_s   = T_opt
36          Ca_s  = Ca_opt
37          Cc_s  = Cc_opt
38
```

```
39          Tc_s  = Tc_opt
40          Fin_s = Fout
41
42          # ========================
43          # Ganancias PI
44          # ========================
45          Kc_Cb   = 3.09
46          tauI_Cb = 0.03
47
48          Kc_Fin   = 0.84
49          tauI_Fin = 1.85
50
51          # ========================
52          # Iniciales DEL SISTEMA
53          # ========================
54          Cb0 = 0.0
55          T0  = 327.0
56          V0  = 80.0
57          Ca0 = 1.0
58          Cc0 = 0.0
59          I_Cb0 = 0.0
60          I_V0  = 0.0
61
62          y0 = [Cb0, T0, V0, Ca0, Cc0, I_Cb0, I_V0]
63
64
65          def CSTR_PI(t, y):
66          Cb, T, V, Ca, Cc, I_Cb, I_V = y
67
68          Veff = max(V, 1e-6)
69          Teff = max(T, 1.0)
70
71          # Errores
72          e_Cb = Cb_s - Cb
73          e_V  = V_s - V
74
75          # Controlador PI sin saturacion
76          Tc  = float(Tc_s  + Kc_Cb  * (e_Cb + I_Cb / tauI_Cb))
77          Fin = float(Fin_s + Kc_Fin * (e_V  + I_V  / tauI_Fin))
78
79          # Cinetica
80          ra = ka * math.exp(-EaR / Teff) * Ca
81          rb = kb * math.exp(-EbR / Teff) * Cb
82
83          # Ecuaciones del reactor
84          dCb = ra - rb - Fin * Cb / Veff
85
```

```
86          dT = (
87          Fin * (Tf - T) / Veff
88          - Ha * ra / (rho * Cp)
89          - Hb * rb / (rho * Cp)
90          + UA * (Tc - T) / (Veff * rho * Cp)
91          )
92
93          dV  = Fin - Fout
94          dCa = Fin * (Cain - Ca) / Veff - ra
95          dCc = rb - Fin * Cc / Veff
96
97          # Integrales del PI
98          dI_Cb = e_Cb
99          dI_V  = e_V
100
101         return [dCb, dT, dV, dCa, dCc, dI_Cb, dI_V]
102
103
104         # ==========================
105         # Simulacion
106         # ==========================
107         t0 = 0.0
108         tf = 25.0
109         t_eval = np.linspace(t0, tf, 100)
110
111         sol = solve_ivp(
112         CSTR_PI,
113         (t0, tf),
114         y0,
115         method="BDF",
116         t_eval=t_eval
117         )
118
119         print(sol.success)
120         print(sol.message)
121
122         # ==========================
123         # Reconstruccion de variables
124         # ==========================
125         t    = sol.t
126         Cb   = sol.y[0]
127         T    = sol.y[1]
128         V    = sol.y[2]
129         Ca   = sol.y[3]
130         Cc   = sol.y[4]
131         I_Cb = sol.y[5]
132         I_V  = sol.y[6]
```

```
133
134         e_Cb = Cb_s - Cb
135         e_V  = V_s - V
136
137         Tc_unsat  = Tc_s  + Kc_Cb  * (e_Cb + I_Cb / tauI_Cb)
138         Fin_unsat = Fin_s + Kc_Fin * (e_V  + I_V  / tauI_Fin)
139
140         Tc  = Tc_unsat
141         Fin = Fin_unsat
142
143         # =========================
144         # Mostrar punto de operacion usado
145         # =========================
146         print("\n=========================")
147         print("PUNTO DE OPERACION USADO")
148         print("=========================")
149         print(f"Tc_s  = {Tc_s:.2f} K")
150         print(f"T_s   = {T_s:.2f} K")
151         print(f"Ca_s  = {Ca_s:.6f}")
152         print(f"Cb_s  = {Cb_s:.6f}")
153         print(f"Cc_s  = {Cc_s:.6f}")
154         print(f"V_s   = {V_s:.2f}")
155         print(f"Fin_s = {Fin_s:.2f}")
156
157         # =========================
158         # Graficas en subplots
159         # =========================
160         fig, axs = plt.subplots(2, 4, figsize=(20, 9))
161
162         # --- Concentraciones ---
163         axs[0, 0].plot(t, Ca, label="Ca")
164         axs[0, 0].plot(t, Cb, label="Cb")
165         axs[0, 0].plot(t, Cc, label="Cc")
166         #axs[0, 0].axhline(Ca_s, linestyle="--", label="Ca_s")
167         axs[0, 0].axhline(Cb_s, linestyle="--", color="red", linewidth
                =0.8, label="Cb_s")
168         #axs[0, 0].axhline(Cc_s, linestyle="--", label="Cc_s")
169         axs[0, 0].set_xlabel("t (s)")
170         axs[0, 0].set_ylabel("Concentracion")
171         axs[0, 0].set_title("Concentraciones")
172         axs[0, 0].grid(True)
173         axs[0, 0].legend()
174
175         # --- Volumen ---
176         axs[0, 1].plot(t, V, label="V")
177         axs[0, 1].axhline(V_s, linestyle="--", label="V_s")
178         axs[0, 1].set_xlabel("t (s)")
```

```
179         axs[0, 1].set_ylabel("Volumen")
180         axs[0, 1].set_title("Volumen")
181         axs[0, 1].grid(True)
182         axs[0, 1].legend()
183 
184         # --- Temperatura del reactor ---
185         axs[0, 2].plot(t, T, label="T")
186         axs[0, 2].axhline(T_s, linestyle="--", label="T_s")
187         axs[0, 2].set_xlabel("t (s)")
188         axs[0, 2].set_ylabel("Temperatura (K)")
189         axs[0, 2].set_title("Temperatura del reactor")
190         axs[0, 2].grid(True)
191         axs[0, 2].legend()
192 
193         # --- Error de Cb ---
194         axs[0, 3].plot(t, e_Cb, label="e_Cb = Cb_s - Cb")
195         axs[0, 3].set_xlabel("t (s)")
196         axs[0, 3].set_ylabel("Error")
197         axs[0, 3].set_title("Error en Cb")
198         axs[0, 3].grid(True)
199         axs[0, 3].legend()
200 
201         # --- Tc en subplot propio ---
202         axs[1, 0].plot(t, Tc, label="Tc")
203         axs[1, 0].axhline(Tc_s, linestyle="--", label="Tc_s")
204         axs[1, 0].set_xlabel("t (s)")
205         axs[1, 0].set_ylabel("Tc (K)")
206         axs[1, 0].set_title("Temperatura de camisa")
207         axs[1, 0].grid(True)
208         axs[1, 0].legend()
209 
210         # --- Fin en subplot propio ---
211         axs[1, 1].plot(t, Fin, label="Fin")
212         axs[1, 1].axhline(Fin_s, linestyle="--", label="Fin_s")
213         axs[1, 1].set_xlabel("t (s)")
214         axs[1, 1].set_ylabel("Fin")
215         axs[1, 1].set_title("Caudal de entrada")
216         axs[1, 1].grid(True)
217         axs[1, 1].legend()
218 
219         # --- Error de volumen ---
220         axs[1, 2].plot(t, e_V, label="e_V = V_s - V")
221         axs[1, 2].set_xlabel("t (s)")
222         axs[1, 2].set_ylabel("Error")
223         axs[1, 2].set_title("Error en V")
224         axs[1, 2].grid(True)
225         axs[1, 2].legend()
```

```
226
227         # --- Integrales ---
228         axs[1, 3].plot(t, I_Cb, label="I_Cb")
229         axs[1, 3].plot(t, I_V, label="I_V")
230         axs[1, 3].set_xlabel("t (s)")
231         axs[1, 3].set_ylabel("Integral")
232         axs[1, 3].set_title("Estados integrales")
233         axs[1, 3].grid(True)
234         axs[1, 3].legend()
235
236         plt.tight_layout()
237         plt.show()
238
239         print("Dimensiones:", sol.y.shape)
```

D. OBTENCIÓN DE DATOS PARA EL ENTRENAMIENTO DE UNA RED NEURONAL.

D.1. Muestreo cambiando los parámetros del controlador

```
1         import numpy as np
2         import pandas as pd
3         import math
4         from scipy.integrate import solve_ivp
5         from scipy.stats import qmc
6         import matplotlib.pyplot as plt
7
8         # ==========================
9         # Parametros del proceso
10        # ==========================
11        Tf    = 350
12        Cain  = 1
13        Fout  = 100
14        rho   = 1000
15        Cp    = 0.239
16        UA    = 5e4
17        Ha    = 5e3
18        EaR   = 8750
19        ka    = 7.2e10
20        Hb    = 4e3
21        EbR   = 10750
22        kb    = 8.2e10
23
24        # ==========================
```

```
25          # Comprobacion basica
26          # ========================
27          if Tc_opt is None:
28          raise ValueError("No existe punto de compromiso. Ejecuta antes
                el codigo de estados estacionarios.")
29
30          # ========================
31          # Setpoints y bias
32          # ========================
33          Cb_s = Cb_opt
34          V_s  = 100.0
35          T_s  = T_opt
36          Ca_s = Ca_opt
37          Cc_s = Cc_opt
38
39          Tc_s  = Tc_opt
40          Fin_s = Fout
41
42          # ========================
43          # Iniciales
44          # ========================
45          Cb0 = 0.0
46          T0  = 327.0
47          V0  = 80.0
48          Ca0 = 1.0
49          Cc0 = 0.0
50          I_Cb0 = 0.0
51          I_V0  = 0.0
52
53          y0 = [Cb0, T0, V0, Ca0, Cc0, I_Cb0, I_V0]
54
55          # ========================
56          # Tiempo
57          # ========================
58          t0 = 0.0
59          tf = 40.0
60          t_eval = np.linspace(t0, tf, 150)
61
62          # ========================================================
63          # RANGOS DE MUESTREO
64          # ========================================================
65
66          param_ranges = {
67              "Kc_Cb":    (0.1, 10.0),
68              "tauI_Cb":  (0.01, 5.0),
69              "Kc_Fin":   (0.1, 10.0),
70              "tauI_Fin": (0.01, 5.0)
```

```
71          }
72
73          param_names = list(param_ranges.keys())
74
75          lower_bounds = np.array([param_ranges[p][0] for p in
                param_names])
76          upper_bounds = np.array([param_ranges[p][1] for p in
                param_names])
77
78          # ========================================================
79          # MUESTREO SOBOL
80          # ========================================================
81
82          m = 11
83          n_samples = 2**m
84
85          sampler = qmc.Sobol(d=4, scramble=True)
86          sample_unit = sampler.random_base2(m=m)
87
88          sample_params = qmc.scale(sample_unit, lower_bounds,
                upper_bounds)
89
90          df_params = pd.DataFrame(sample_params, columns=param_names)
91
92          print("Numero de simulaciones:", len(df_params))
93          display(df_params.head())
94
95          # ========================================================
96          # MODELO CSTR + PI
97          # ========================================================
98
99          def CSTR_PI(t, y, Kc_Cb, tauI_Cb, Kc_Fin, tauI_Fin):
100
101         Cb, T, V, Ca, Cc, I_Cb, I_V = y
102
103         # Solo proteccion matematica para evitar division por cero
104         Veff = V
105         Teff = T
106
107         # Errores
108         e_Cb = Cb_s - Cb
109         e_V  = V_s - V
110
111         # Controlador PI completamente libre
112         Tc  = float(Tc_s  + Kc_Cb  * (e_Cb + I_Cb / tauI_Cb))
113         Fin = float(Fin_s + Kc_Fin * (e_V  + I_V  / tauI_Fin))
114
```

```
115         # Cinetica
116         ra = ka * math.exp(-EaR / Teff) * Ca
117         rb = kb * math.exp(-EbR / Teff) * Cb
118
119         # Ecuaciones del reactor
120         dCb = ra - rb - Fin * Cb / Veff
121
122         dT = (
123         Fin * (Tf - T) / Veff
124         - Ha * ra / (rho * Cp)
125         - Hb * rb / (rho * Cp)
126         + UA * (Tc - T) / (Veff * rho * Cp)
127         )
128
129         dV  = Fin - Fout
130         dCa = Fin * (Cain - Ca) / Veff - ra
131         dCc = rb - Fin * Cc / Veff
132
133         # Integrales del PI
134         dI_Cb = e_Cb
135         dI_V  = e_V
136
137         return [dCb, dT, dV, dCa, dCc, dI_Cb, dI_V]
138
139         # ========================================================
140         # SIMULACION MASIVA
141         # ========================================================
142
143         dataset_list = []
144         failed_sims = []
145
146         for sim_id, row in df_params.iterrows():
147
148         Kc_Cb    = row["Kc_Cb"]
149         tauI_Cb  = row["tauI_Cb"]
150         Kc_Fin   = row["Kc_Fin"]
151         tauI_Fin = row["tauI_Fin"]
152
153         try:
154
155         sol = solve_ivp(
156         CSTR_PI,
157         (t0, tf),
158         y0,
159         args=(Kc_Cb, tauI_Cb, Kc_Fin, tauI_Fin),
160         method="BDF",
161         t_eval=t_eval,
```

```
        rtol=1e-6,
        atol=1e-8
        )

        if not sol.success:
        failed_sims.append((sim_id, sol.message))
        continue

        if sol.y.shape[1] != len(t_eval):
        failed_sims.append((sim_id, "Simulacion incompleta"))
        continue

        t     = sol.t
        Cb    = sol.y[0]
        T     = sol.y[1]
        V     = sol.y[2]
        Ca    = sol.y[3]
        Cc    = sol.y[4]
        I_Cb  = sol.y[5]
        I_V   = sol.y[6]

        e_Cb  = Cb_s  - Cb
        e_V   = V_s   - V

        # Reconstruccin de acciones PI sin saturacin
        Tc   = Tc_s   + Kc_Cb   * (e_Cb + I_Cb / tauI_Cb)
        Fin  = Fin_s  + Kc_Fin  * (e_V  + I_V  / tauI_Fin)

        df_sim = pd.DataFrame({
            "sim_id": sim_id,
            "t": t,

            "Ca": Ca,
            "Cb": Cb,
            "Cc": Cc,
            "T": T,
            "V": V,

            "Tc": Tc,
            "Fin": Fin,

            "I_Cb": I_Cb,
            "I_V": I_V,

            "e_Cb": e_Cb,
            "e_V": e_V,

```

```
            "Kc_Cb": Kc_Cb,
            "tauI_Cb": tauI_Cb,
            "Kc_Fin": Kc_Fin,
            "tauI_Fin": tauI_Fin,


        })

        dataset_list.append(df_sim)

        except Exception as e:
        failed_sims.append((sim_id, str(e)))

        # ============================================================
        # DATASET FINAL
        # ============================================================

        dataset_long = pd.concat(dataset_list, ignore_index=True)

        print("Simulaciones correctas:", dataset_long["sim_id"].
            nunique())
        print("Simulaciones fallidas:", len(failed_sims))
        print("Filas totales:", len(dataset_long))

        display(dataset_long.head())
```

D.2. Espacio ocupado de los parámetros del controlador.

```
        fig, axes = plt.subplots(1, 2, figsize=(14, 6))

    # --- Controlador Cb ---
    axes[0].scatter(df_params["Kc_Cb"], df_params["tauI_Cb"], s=10)
    axes[0].set_xlabel(r"$K_{c,Cb}$")
    axes[0].set_ylabel(r"$\tau_{I,Cb}$")
    axes[0].set_title("Muestreo Sobol: controlador Cb")
    axes[0].grid(False)

    # --- Controlador V ---
    axes[1].scatter(df_params["Kc_Fin"], df_params["tauI_Fin"], s=10)
    axes[1].set_xlabel(r"$K_{c,Fin}$")
    axes[1].set_ylabel(r"$\tau_{I,Fin}$")
    axes[1].set_title("Muestreo Sobol: controlador V")
    axes[1].grid(False)

    plt.tight_layout()
    plt.show()
```

D.3. Filtrado y descarte

```
        # ======================================================
        # FILTRADO DE SIMULACIONES QUE ALCANZAN ESTADO ESTACIONARIO
        # ======================================================

        steady_vars = ["Ca", "Cb", "Cc", "T", "V", "Tc", "Fin"]

        setpoints = {
            "Ca":  Ca_s,
            "Cb":  Cb_s,
            "Cc":  Cc_s,
            "T":   T_s,
            "V":   V_s,
            "Tc":  Tc_s,
            "Fin": Fin_s
        }

        # Tolerancias relativas
        tolerances = {
            "Ca":  0.03 * abs(Ca_s),
            "Cb":  0.03 * abs(Cb_s),
            "Cc":  0.03 * abs(Cc_s),
            "T":   0.03 * abs(T_s),
            "V":   0.03 * abs(V_s),
            "Tc":  0.03 * abs(Tc_s),
            "Fin": 0.03 * abs(Fin_s)
        }

        # Tolerancias minimas para evitar problemas si algun setpoint
            es muy pequeno
        min_tolerances = {
            "Ca":  1e-4,
            "Cb":  1e-4,
            "Cc":  1e-4,
            "T":   1e-3,
            "V":   1e-3,
            "Tc":  1e-3,
            "Fin": 1e-3
        }

        for var in steady_vars:
        tolerances[var] = max(tolerances[var], min_tolerances[var])

```

```
42        # Tiempo minimo continuo dentro del setpoint
43        min_stable_time = 3.0
44
45
46        def reaches_steady_state(df_sim, steady_vars, setpoints,
              tolerances, min_stable_time):
47        """
48        Una simulacion es valida solo si TODAS las variables
              permanecen
49        dentro de su tolerancia durante al menos min_stable_time
              segundos continuos.
50        """
51
52        df_sim = df_sim.sort_values("t").reset_index(drop=True)
53        t = df_sim["t"].values
54
55        inside_all = np.ones(len(df_sim), dtype=bool)
56
57        for var in steady_vars:
58        error_abs = np.abs(df_sim[var].values - setpoints[var])
59        inside_var = error_abs <= tolerances[var]
60
61        # Si una variable no esta dentro de tolerancia,
62        # la condicion global deja de cumplirse
63        inside_all = inside_all & inside_var
64
65        start_idx = None
66
67        for i in range(len(inside_all)):
68
69        if inside_all[i] and start_idx is None:
70        start_idx = i
71
72        elif not inside_all[i] and start_idx is not None:
73        duration = t[i - 1] - t[start_idx]
74
75        if duration >= min_stable_time:
76        return True
77
78        start_idx = None
79
80        # Si la zona estable llega hasta el final de la simulacion
81        if start_idx is not None:
82        duration = t[-1] - t[start_idx]
83
84        if duration >= min_stable_time:
85        return True
```

```
86
87          return False
88
89
90          # ========================================================
91          # APLICAR FILTRO A TODAS LAS SIMULACIONES
92          # ========================================================
93
94          valid_sim_ids = []
95          invalid_sim_ids = []
96
97          for sim_id, df_sim in dataset_long.groupby("sim_id"):
98
99          is_valid = reaches_steady_state(
100         df_sim=df_sim,
101         steady_vars=steady_vars,
102         setpoints=setpoints,
103         tolerances=tolerances,
104         min_stable_time=min_stable_time
105         )
106
107         if is_valid:
108         valid_sim_ids.append(sim_id)
109         else:
110         invalid_sim_ids.append(sim_id)
111
112
113         # ========================================================
114         # DATASET FILTRADO FINAL
115         # ========================================================
116
117         dataset = dataset_long[dataset_long["sim_id"].isin(
                valid_sim_ids)].copy()
118
119         dataset_descartado = dataset_long[dataset_long["sim_id"].isin(
                invalid_sim_ids)].copy()
120
121
122         print("================================================")
123         print("FILTRADO DE ESTADO ESTACIONARIO")
124         print("================================================")
125         print(f"Simulaciones totales en dataset_long: {dataset_long['
                sim_id'].nunique()}")
126         print(f"Simulaciones validas guardadas en dataset: {dataset['
                sim_id'].nunique()}")
127         print(f"Simulaciones descartadas: {len(invalid_sim_ids)}")
128         print("================================================")
```

```
129
130         print("IDs de simulaciones descartadas:")
131         print(invalid_sim_ids)
132
133         display(dataset.head())
```

D.4. Cambio de la estructura del Dataset.

```
1
2
3         # ============================================================
4         # VARIABLES EN LA TRAYECTORIA
5         # ============================================================
6
7         trajectory_vars = ["Ca","Cb", "Cc", "T", "V", "Tc", "Fin", "
              I_Cb", "I_V", "e_Cb", "e_V" ]
8
9         # ============================================================
10        # CREAR DATASET WIDE
11        # 1 fila = 1 simulacion completa
12        # ============================================================
13
14        wide_rows = []
15
16        for sim_id in dataset["sim_id"].unique():
17
18        df_sim = dataset[dataset["sim_id"] == sim_id].sort_values("t")
19
20        row = {}
21
22        # ------------------------------------------------------
23        # INPUTS (constantes en cada simulacion)
24        # ------------------------------------------------------
25
26        row["Kc_Cb"]     = df_sim["Kc_Cb"].iloc[0]
27        row["tauI_Cb"]   = df_sim["tauI_Cb"].iloc[0]
28        row["Kc_Fin"]    = df_sim["Kc_Fin"].iloc[0]
29        row["tauI_Fin"] = df_sim["tauI_Fin"].iloc[0]
30
31        # ------------------------------------------------------
32        # OUTPUTS (trayectorias completas)
33        # ------------------------------------------------------
34
35        for var in trajectory_vars:
36
37        values = df_sim[var].values
```

```
38
39          for i, value in enumerate(values):
40
41          row[f"{var}_t{i}"] = value
42
43          wide_rows.append(row)
44
45          # ============================================================
46          # DATASET FINAL WIDE
47          # ============================================================
48
49          dataset_wide = pd.DataFrame(wide_rows)
50
51          # ============================================================
52          # VERIFICACION
53          # ============================================================
54
55          print(dataset_wide.shape)
56
57          dataset_wide.head()
```

D.5. Espacio ocupado de los parámetros del controlador.

```
1           fig, axes = plt.subplots(1, 2, figsize=(14, 6))
2
3           # Tamanos de fuente
4           title_size = 18   # tamano de los titulos
5           label_size = 16   # tamano de las etiquetas de los ejes
6           tick_size  = 14   # tamano de los numeros de los ejes
7
8           # --- Controlador Cb ---
9           axes[0].scatter(df_params["Kc_Cb"], df_params["tauI_Cb"], s
                =10)
10          axes[0].set_xlabel(r"$K_{c,1}$", fontsize=label_size)
11          axes[0].set_ylabel(r"$\tau_{I,1}$", fontsize=label_size)
12          axes[0].set_title(r"Controlador $C_B$", fontsize=title_size)
13          axes[0].tick_params(axis='both', labelsize=tick_size)
14          axes[0].grid(False)
15
16          # --- Controlador V ---
17          axes[1].scatter(df_params["Kc_Fin"], df_params["tauI_Fin"], s
                =10)
18          axes[1].set_xlabel(r"$K_{c,2}$", fontsize=label_size)
19          axes[1].set_ylabel(r"$\tau_{I,2}$", fontsize=label_size)
20          axes[1].set_title(r"Controlador $V$", fontsize=title_size)
21          axes[1].tick_params(axis='both', labelsize=tick_size)
```

```
22          axes[1].grid(False)
23
24          plt.tight_layout()
25          plt.show()
26
27
28          VARIABLES CRUZADAS
29
30
31          fig, axes = plt.subplots(1, 2, figsize=(14, 6))
32
33          # Tamanos de fuente
34          title_size = 18
35          label_size = 16
36          tick_size  = 14
37
38          # --- Ganancias proporcionales ---
39          axes[0].scatter(df_params["Kc_Cb"], df_params["Kc_Fin"], s=10)
40          axes[0].set_xlabel(r"$K_{c,1}$", fontsize=label_size)
41          axes[0].set_ylabel(r"$K_{c,2}$", fontsize=label_size)
42          axes[0].set_title(r"Ganancias", fontsize=title_size)
43          axes[0].tick_params(axis='both', labelsize=tick_size)
44          axes[0].grid(False)
45
46          # --- Tiempos integrales ---
47          axes[1].scatter(df_params["tauI_Cb"], df_params["tauI_Fin"], s
               =10)
48          axes[1].set_xlabel(r"$\tau_{I,1}$", fontsize=label_size)
49          axes[1].set_ylabel(r"$\tau_{I,2}$", fontsize=label_size)
50          axes[1].set_title(r"Tiempos integrales", fontsize=title_size)
51          axes[1].tick_params(axis='both', labelsize=tick_size)
52          axes[1].grid(False)
53
54          plt.tight_layout()
55          plt.show()
56
57
58
59          COMPARATIVA SIMULACIONES GUARDADAS Y DESCARTADAS
60
61          # ======================================================
62          # GRAFICO: PARAMETROS DEL CONTROLADOR Cb
63          # GUARDADAS VS DESCARTADAS
64          # ======================================================
65
66          plt.figure(figsize=(7, 6))
67
```

```
68          # Tamanos de fuente
69          title_size = 18
70          label_size = 16
71          tick_size = 14
72          legend_size = 13
73
74          # Simulaciones descartadas
75          plt.scatter(
76          df_descartadas["Kc_Cb"],
77          df_descartadas["tauI_Cb"],
78          s=20,
79          alpha=0.5,
80          color="red",
81          label="Simulaciones descartadas"
82          )
83
84          # Simulaciones guardadas
85          plt.scatter(
86          df_guardadas["Kc_Cb"],
87          df_guardadas["tauI_Cb"],
88          s=20,
89          alpha=0.8,
90          color="blue",
91          label="Simulaciones guardadas"
92          )
93
94          # Etiquetas y formato
95          plt.xlabel(r"$K_{c,1}$", fontsize=label_size)
96          plt.ylabel(r"$\tau_{I,1}$", fontsize=label_size)
97          plt.title(r"Controlador $C_B$", fontsize=title_size)
98          plt.xticks(fontsize=tick_size)
99          plt.yticks(fontsize=tick_size)
100         plt.legend(fontsize=legend_size)
101         plt.grid(False)
102
103         plt.tight_layout()
104         plt.show()
```

D.6. Gráficos de variabilidad

```
1           # ======================================================
2           # DATOS NECESARIOS PARA LOS GRAFICOS DE VARIABILIDAD
3           # ======================================================
4
5           variables = ["Cb", "T", "V"]
6
```

```
        dataset = dataset.sort_values(["sim_id", "t"]).reset_index(
            drop=True)

        analysis_data = {}

        for var in variables:

        # Matriz: filas = simulaciones, columnas = tiempo
        pivot = dataset.pivot(index="sim_id", columns="t", values=var)

        # Eliminar simulaciones incompletas
        pivot = pivot.dropna()

        X = pivot.values
        t = pivot.columns.values

        # Desviacion estandar entre simulaciones en cada instante de
            tiempo
        std_over_time = np.std(X, axis=0)

        analysis_data[var] = {
            "t": t,
            "X": X,
            "std_over_time": std_over_time
        }


        # ========================================================
        # GRAFICOS DE VARIABILIDAD
        # ========================================================

        for var in variables:

        data = analysis_data[var]
        t = data["t"]
        X = data["X"]
        std_over_time = data["std_over_time"]


        mean_traj = np.mean(X, axis=0)
        min_traj = np.min(X, axis=0)
        max_traj = np.max(X, axis=0)

        fig, axes = plt.subplots(1, 2, figsize=(12, 4))

        # -------------------------------
        # 1) Banda de trayectorias
```

```
        # --------------------------------
        axes[0].fill_between(t, min_traj, max_traj, alpha=0.25)
        axes[0].plot(t, mean_traj, linewidth=2, label="Media")
        axes[0].set_title(f"{var}: rango de trayectorias")
        axes[0].set_xlabel("Tiempo [s]")
        axes[0].set_ylabel(var)
        axes[0].legend()
        axes[0].grid(False)

        # --------------------------------
        # 2) Desviacion estandar temporal
        # --------------------------------
        axes[1].plot(t, std_over_time, linewidth=2)
        axes[1].set_title(f"{var}: std entre simulaciones")
        axes[1].set_xlabel("Tiempo [s]")
        axes[1].set_ylabel("Std")
        axes[1].grid(False)


        plt.tight_layout()
        plt.show()
```

E. RED NEURONAL

E.1. Importación de librerías y cargado del Dataset

```

        # LIBRERIAS IMPORTADAS
        import os
        import pandas as pd
        import numpy as np

        import tensorflow as tf
        from tensorflow import keras

        import matplotlib.pyplot as plt
        from google.colab import files

        print(tf.__version__)


        #CARGAR EL DATASET

        from google.colab import files
```

```
uploaded = files.upload()  #se debera descargar previamente despues del filtrado de datos y cambio de estructura

#COLUMNAS INNECESARIAS
dataset_wide = dataset_wide.drop(columns=['sim_id_original'])
```

E.2. Separación del Dataset en datos de entrenamiento y de prueba

```
from sklearn.model_selection import train_test_split

# ============================================================
# INPUTS
# ============================================================

input_cols = [
"Kc_Cb",
"tauI_Cb",
"Kc_Fin",
"tauI_Fin"
]

input = dataset_wide[input_cols].values #datset_wide es en este caso el nombre del Dataset

# ============================================================
# OUTPUTS
# ============================================================

output = dataset_wide.drop(columns=input_cols).values

# ============================================================
# TRAIN / TEST
# ============================================================

input_train, input_test, output_train, output_test = train_test_split(input,
                output,
                test_size=0.15,
                random_state=42)
```

E.3. Normalización

```
import pandas as pd

# ============================================================
# INPUTS
# ============================================================

input_cols = [
"Kc_Cb",          #Kc_1
"tauI_Cb",        #tauI_1
"Kc_Fin",
"tauI_Fin"
]

# ============================================================
# OUTPUTS
# ============================================================

output_cols = [col for col in dataset_wide.columns if col not
    in input_cols]

# ============================================================
# TRAIN DATAFRAMES
# ============================================================

input_train_df = pd.DataFrame(input_train, columns=input_cols)
input_test_df  = pd.DataFrame(input_test,  columns=input_cols)

output_train_df = pd.DataFrame(output_train, columns=
    output_cols)
output_test_df  = pd.DataFrame(output_test,  columns=
    output_cols)

# ============================================================
# ESTADISTICAS TRAIN INPUT
# ============================================================

train_stats_input = input_train_df.describe().transpose()

def norm_input(x):
return (x - train_stats_input['mean']) / train_stats_input['
    std']

# ============================================================
# ESTADISTICAS TRAIN OUTPUT
# ============================================================

train_stats_output = output_train_df.describe().transpose()
```

```
44
45          def norm_output(x):
46          return (x - train_stats_output['mean']) / train_stats_output['
                std']
47
48          # ============================================================
49          # NORMALIZACION
50          # ============================================================
51
52          input_train_norm = norm_input(input_train_df)
53          input_test_norm  = norm_input(input_test_df)
54
55          output_train_norm = norm_output(output_train_df)
56          output_test_norm  = norm_output(output_test_df)
```

E.4. Definición del modelo.

```
1           from tensorflow.keras import Sequential
2           from tensorflow.keras.layers import Dense, Dropout
3           from tensorflow.keras import regularizers
4
5           def build_model():
6               model = Sequential()
7
8               model.add(Dense(
9               128,
10              activation='relu',
11              kernel_regularizer=regularizers.l2(0.0001),
12              input_shape=(input_train_norm.shape[1],)
13              ))
14
15              model.add(Dense(
16              64,
17              activation='relu',
18              kernel_regularizer=regularizers.l2(0.0001)
19              ))
20
21
22              model.add(Dense(output_train_norm.shape[1]))
23
24          return model
25
26          model = build_model()
```

E.5. Configuración del modelo.

```
        from IPython.core import history

        #  Optimizador
        optimizer = tf.keras.optimizers.Adam(learning_rate=0.0001)

        #  Compilacion
        model.compile(
        loss='mae',
        optimizer=optimizer,
        metrics=['mae']
        )
```

E.6. Entrenamiento del modelo.

```


    #Entrenamiento
    history = model.fit(
    input_train_norm,
    output_train_norm,
    validation_split=0.2,
    verbose=1,
    epochs=1000,
    callbacks=[early_stop, lr_scheduler]
    )









    #  Early stopping
    early_stop = tf.keras.callbacks.EarlyStopping(
    monitor='val_loss',
    patience=5,
    restore_best_weights=True
    )

    #  Scheduler de learning rate
    lr_scheduler = tf.keras.callbacks.ReduceLROnPlateau(
    monitor='val_loss',
    factor=0.5,
    patience=5,
    min_lr=1e-6,
    verbose=1
    )

```

```
31
32      # REPRESENTACION FIGURA 29
33      def plot_history(history):
34
35      hist = pd.DataFrame(history.history)
36      hist['epoch'] = history.epoch
37
38      plt.figure(figsize=(8,5))
39
40      plt.xlabel('Epoch')
41      plt.ylabel('MAE')
42
43      plt.plot(
44      hist['epoch'],
45      hist['mae'],
46      'r--',
47      label='Training Error'
48      )
49
50      plt.plot(
51      hist['epoch'],
52      hist['val_mae'],
53      'b',
54      label='Validation Error'
55      )
56
57      plt.ylim([0, 0.6])
58
59      plt.legend()
60      plot_history(history)
```

E.7. Evaluación con los datos de prueba.

```
1       loss, mae = model.evaluate(
2                       input_test_norm,
3                       output_test_norm,
4                       verbose=2
5                       )
6
7                       print("Test Loss (MSE):", loss)
8                       print("Test MAE:", mae)
```

E.8. Predicciones.

```
1
2       y_pred = model.predict(input_test_norm)
3
4
5
6
7       #OBTENCION DE GRAFICAS
8
9       # ============================================================
10      # 1) USAR SOLO LAS COLUMNAS QUE QUEDARON TRAS QUITAR NaN
11      # ============================================================
12
13      output_cols_clean = output_test_norm.columns
14
15      stats_mean = train_stats_output.loc[output_cols_clean, "mean"
            ].values
16      stats_std  = train_stats_output.loc[output_cols_clean, "std"].
            values
17
18
19      # ============================================================
20      # 2) DESNORMALIZAR
21      # ============================================================
22
23      output_test_real = output_test_norm.values * stats_std +
            stats_mean
24      output_pred_real = y_pred * stats_std + stats_mean
25
26
27      # ============================================================
28      # 3) SELECCIONAR SIMULACION
29      # ============================================================
30
31      sim_index = 4   # Cambia aquI el numero de simulacion
32
33
34      # ============================================================
35      # 4) FUNCION PARA EXTRAER VARIABLES
36      # ============================================================
37
38      def extract_variable(data, var):
39      cols_var = [
40      i for i, col in enumerate(output_cols_clean)
41      if col.startswith(var + "_t")
42      ]
43      return data[sim_index, cols_var]
```

```
44
45
46          # ============================================================
47          # 5) EXTRAER Cb, T Y V
48          # ============================================================
49
50          Cb_real = extract_variable(output_test_real, "Cb")
51          Cb_pred = extract_variable(output_pred_real, "Cb")
52
53          T_real = extract_variable(output_test_real, "T")
54          T_pred = extract_variable(output_pred_real, "T")
55
56          V_real = extract_variable(output_test_real, "V")
57          V_pred = extract_variable(output_pred_real, "V")
58
59
60          # ============================================================
61          # 6) VECTOR DE TIEMPO
62          # ============================================================
63
64          n_time = len(Cb_real)
65          t = np.linspace(0, 40, n_time)
66          # ============================================================
67          # 6) GRAFICOS
68          # ============================================================
69
70          # Colores
71          color_pred = "#0000CD"   # Verde claro
72          color_Cb   = "#D62728"   # Naranja
73          color_T    = "#D62728"   # Rojo
74          color_V    = "#D62728"   # Gris
75
76          # Grosor de lineas
77          lw_real = 1.8
78          lw_pred = 1.5
79
80          fig, axes = plt.subplots(1, 3, figsize=(18, 5))
81
82          # ------------------------------------------------------------
83          # C_B
84          # ------------------------------------------------------------
85          axes[0].plot(
86          t, Cb_real,
87          color=color_Cb,
88          linewidth=lw_real,
89          label="$C_B$ real"
90          )
```

```
91
92          axes[0].plot(
93          t, Cb_pred,
94          "--",
95          color=color_pred,
96          linewidth=lw_pred,
97          label="$C_B$ predicho"
98          )
99
100         axes[0].set_xlabel("t (s)")
101         axes[0].set_ylabel("$C_B$ $(mol/m^3)$")
102         axes[0].set_title("$C_B$")
103         axes[0].legend()
104         axes[0].grid(False)
105
106         # ------------------------------------------------------------
107         # T
108         # ------------------------------------------------------------
109         axes[1].plot(
110         t, T_real,
111         color=color_T,
112         linewidth=lw_real,
113         label="T real"
114         )
115
116         axes[1].plot(
117         t, T_pred,
118         "--",
119         color=color_pred,
120         linewidth=lw_pred,
121         label="T predicho"
122         )
123
124         axes[1].set_xlabel("t (s)")
125         axes[1].set_ylabel("T (K)")
126         axes[1].set_title("T")
127         axes[1].legend()
128         axes[1].grid(False)
129
130         # ------------------------------------------------------------
131         # V
132         # ------------------------------------------------------------
133         axes[2].plot(
134         t, V_real,
135         color=color_V,
136         linewidth=lw_real,
137         label="V real"
```

```
        )

        axes[2].plot(
        t, V_pred,
        "--",
        color=color_pred,
        linewidth=lw_pred,
        label="V predicho"
        )

        axes[2].set_xlabel("t (s)")
        axes[2].set_ylabel("V $(m^3)$")
        axes[2].set_title("V")
        axes[2].legend()
        axes[2].grid(False)



        from google.colab import files

        # Guardar cada eje (subplot) por separado
        for i, ax in enumerate(fig.axes):

        # Obtener el bounding box del subplot
        extent = ax.get_tightbbox(fig.canvas.get_renderer())
        extent = extent.transformed(fig.dpi_scale_trans.inverted())

        # Nombre del archivo
        nombre_archivo = f"prediccion_{i+1}.png"

        # Guardar unicamente ese subplot
        fig.savefig(
        nombre_archivo,
        dpi=300,
        bbox_inches=extent
        )

        # Descargar automaticamente
        #files.download(nombre_archivo)

        plt.tight_layout()
```